

Efficient Polynomial System Solving via Dixon Resultants: Applications to AO Primitives

Haohai Suo^{1,2,3} and Jiamin Cui^{1,2,3}(✉)

¹ School of Cyber Science and Technology, Shandong University, Qingdao, Shandong, China

² State Key Laboratory of Cryptography and Digital Economy Security, Shandong University, Qingdao, 266237, Shandong, China

³ Key Laboratory of Cryptologic Technology and Information Security, Ministry of Education, Shandong University, Jinan, China
haohai.suo@mail.sdu.edu.cn, cuijiamin@sdu.edu.cn

Abstract. Solving multivariate polynomial systems is a fundamental problem in cryptanalysis, with increasing relevance in algebraic attacks on arithmetization-oriented (AO) primitives. Current approaches include Gröbner bases and the Sylvester resultant. However, Gröbner basis methods typically rely on FGLM to change the order, which requires a zero-dimensional ideal, whereas the Sylvester resultant eliminates only one variable at a time, limiting its flexibility in multivariate elimination.

We revisit the Dixon resultant as an efficient and flexible tool for eliminating several variables simultaneously. We derive refined upper bounds on the Dixon matrix size via lattice-path counting and analyze the complexity under several determinant computation models, yielding explicit complexity estimates. For well-determined systems, the Dixon resultant is a viable alternative to Gröbner basis methods. Moreover, given their respective limitations, Dixon resultants are particularly suitable for parametric elimination in underdetermined systems, while Gröbner basis methods are generally preferable for solving overdetermined systems.

We present an efficient open-source C implementation, **DRSolve**, with multiple determinant methods and a degree-aware submatrix selection strategy to mitigate the impact of extraneous factors. On the randomly generated well-determined systems tested, our implementation is competitive with the state-of-the-art Gröbner basis solvers Magma and msolve, with **DRSolve** performing better in the low-variable/high-degree regime, while Magma and msolve remain preferable in the high-variable/low-degree regime.

Finally, we formulate three elimination strategies for polynomial systems arising from AO primitives: direct elimination, iterative elimination, and reduction-based hybrid elimination. We demonstrate these strategies on **POSEIDON**, **Vision**, and **XHash12**, obtaining complexity reductions in many cases.

Keywords: Dixon resultant, polynomial system solving, algebraic cryptanalysis, arithmetization-oriented primitives

1 Introduction

Solving multivariate polynomial systems over finite fields is a basic task in computational algebra. Approaches to this problem include Gröbner basis methods [76,35,34], resultant-based elimination [31,65,56], and characteristic-set methods in the spirit of Wu’s method [83]. In contemporary algebraic cryptanalysis, most attention is paid to Gröbner basis techniques [71,58], while resultant methods—especially the Dixon resultant as a compact multivariate elimination tool—remain much less explored in the algebraic cryptanalysis literature.

These problems are especially relevant for *arithmetization-oriented* (AO) primitives, whose round functions are designed to admit compact arithmetic representations in zero-knowledge (ZK) proof systems, MPC, and homomorphic settings. AO primitives differ substantially from classical binary-oriented designs. Instead of working over small binary fields, they are typically defined over large prime fields or extension fields, and their nonlinear layers are optimized for arithmetic circuits. Following this design philosophy, ZK-friendly AO primitives are commonly organized into three families (Types I–III):⁴

- *Type I (low degree)*: MiMC [2], GMiMC [1], POSEIDON [42], POSEIDON2 [43], Neptune [45];
- *Type II (equivalence relations)*: Vision/Rescue [4], XHash12 [5], Anemoi [25], Arion [70], Griffin [39];
- *Type III (lookup tables)*: Reinforced Concrete [40], Monolith [41], Tip5 [78].

In all three settings, algebraic preimage and collision attacks against these primitives reduce to *constrained-input constrained-output* [19] (CICO) problems, which in turn amount to solving induced polynomial systems whose complexity dominates the security analysis.

Related work. Solving-based algebraic attacks have seen continuous refinement over the last several years. A major line of work [71,24,15,74,58,64,44,7] studies Gröbner basis attacks and, in particular, how to obtain better algebraic models or tighter upper bounds for their complexity on structured AO systems. In [58], the authors refined the complexity analysis by providing more precise estimates for the individual stages of the attack. However, in some cases the upper bounds obtained from such generic Gröbner basis analyses remain inaccurate. For example, the *FreeLunch* approach [15] uses dedicated weighted monomial orderings for which a given polynomial system is already a Gröbner basis, allowing for better complexity bounds.

Another line of work brings classical elimination theory into cryptanalysis. Yang et al. [84] introduced iterative pairwise resultant attacks based on the Sylvester resultant, eliminating one variable at a time from two equations. Bariant et al. [14] further improved this paradigm by exploiting structure inside the polynomial system and performing degree reductions during elimination.

⁴ See the STAP Zoo taxonomy of ZK-friendly designs: <https://stap-zoo.com/zk/>.

These are important developments, but they remain rooted in iterated two-polynomial resultants rather than *multipolynomial* resultants, so the elimination order is comparatively rigid.

Dixon resultant. The Dixon resultant, introduced by A.L. Dixon in 1909 [33], offers a different determinant-based elimination mechanism: it can eliminate several variables simultaneously from a polynomial system and thus serves as a compact multipolynomial resultant in our setting. Compared with the Macaulay resultant [65], the Dixon construction often leads to substantially smaller matrices while preserving the essential elimination information. Kapur et al. [56] also explained how to handle the non-square situation via maximal-rank submatrices, which makes the method viable beyond the classical idealized setting. Tang and Feng [79] turned this construction into a solver by retaining one variable as a parameter. Albrecht [3] implemented a modified DR algorithm for the Courtois Toy Cipher. Our direct method follows this workflow, augmented with degree-aware submatrix selection and a fast polynomial-matrix determinant stage. Subsequently, numerous works [61,85,32,69,68,52,51] studied both the algebraic and algorithmic aspects of Dixon matrices.

For complexity analysis, the size of the Dixon matrix is the first key parameter. Kapur and Saxena [54] gave Newton-polytope and volume-based bounds for unmixed supports, while Chtcherba and Kapur developed multivariate methods for mixed supports [27] and exact size formulas for generic unmixed bivariate systems [28]. We derive an explicit ordered total-degree formula through lattice paths and a Hessenberg determinant, and prove generic attainment. For equal degrees, this extends the quadratic Catalan count of Tang and Feng [79]. A second key parameter is the cost of polynomial-matrix determinants themselves, as expansion by minors [38], Bareiss elimination [48], Kronecker substitution [63], Hermite normal form [60], evaluation-interpolation [48], and sparse interpolation [49] can lead to substantially different complexity estimates, and we accordingly develop a stage-wise complexity model that compares all of them.

Our contributions This paper studies Dixon resultants as a method for *solving polynomial systems*, with AO primitives as a representative application domain. Our main contributions are as follows.

1. We derive refined bounds on Dixon matrix size for mixed systems via lattice-path counting. For uniform-degree systems, the bound specializes to a Fuss–Catalan number [47,6]; for general mixed systems, we obtain an explicit Hessenberg determinant formula, which is attained by the monomial support of generic instances.
2. We develop a stage-wise complexity model for Dixon elimination by comparing multiple determinant computation strategies, yielding a more faithful description of polynomial-matrix behavior.
3. We implement an efficient C library `DRSolve`⁵ supporting multiple determinant routines and a degree-aware submatrix heuristic. Experiments validate

⁵ <https://drsolve.github.io>

the proposed Dixon-matrix bounds and show competitiveness with Magma and msolve on well-determined random systems, with advantages in the low-variable/high-degree regime, while Magma and msolve remain superior in the high-variable/low-degree regime.

4. We organize Dixon-based elimination for AO-induced systems into three strategies: direct, iterative, and hybrid reduction. We illustrate them on POSEIDON, **Vision**, and **XHash12**, respectively, providing concrete algebraic complexity estimates.

Outline. Section 2 reviews the Dixon construction and the determinant computation models used in the paper. Section 3 develops the refined Dixon-matrix bounds and the stepwise complexity analysis. Section 4 discusses implementation-oriented improvements. Section 5 presents the three application patterns on POSEIDON, **Vision**, and **XHash12**.

2 Preliminaries

Notation. Let $\mathbb{K}$ be a field with algebraic closure $\overline{\mathbb{K}}$. Throughout, $\mathcal{F} = \{f_1, \dots, f_n\} \subset \mathbb{K}[\mathbf{a}][\mathbf{x}]$ is a system of n equations in $n - 1 + k$ variables, viewed as polynomials in the eliminated variables $\mathbf{x} = (x_1, \dots, x_{n-1})$ with coefficients in $\mathbb{K}[\mathbf{a}]$, where $\mathbf{a} = (a_1, \dots, a_k)$ are the retained parameters. The total degree of f is denoted by $\deg(f)$ and its degree in x_i by $\deg_{x_i}(f)$; $\langle \mathcal{G} \rangle$ denotes the ideal generated by a set $\mathcal{G}$ of polynomials. All notation is summarized in Table 1.

Table 1. Notation used throughout the paper.

Symbol	Meaning
$\mathcal{F}$	Polynomial System $\mathcal{F} = \{f_1, \dots, f_n\} \subset \mathbb{K}[\mathbf{a}][\mathbf{x}]$
$\mathbf{x}, \overline{\mathbf{x}}, \mathbf{a}$	$n - 1$ eliminated variables, their dual variables, k retained parameters
$X, \overline{X}$	row vectors of monomials in $\mathbf{x}$ and $\overline{\mathbf{x}}$, respectively
$C, \widehat{C}$	cancellation matrix (1) and its refined form (3)
$\Delta_{\mathcal{F}}, \mathcal{M}$	Dixon polynomial (2) and Dixon matrix (Step 2), $\Delta_{\mathcal{F}} = X\mathcal{M}\overline{X}^T$
$D(\mathcal{F})$	Dixon matrix size of $\mathcal{F}$: the maximum of its row and column counts
$\mathcal{M}', \rho$	maximal-rank square submatrix of $\mathcal{M}$, of size $\rho = \text{rank } \mathcal{M} \leq D(\mathcal{F})$
$\mathcal{I}, J, d_{\mathcal{I}}$	$\langle \mathcal{F} \rangle$; elimination ideal $\mathcal{I} \cap \mathbb{K}[\mathbf{a}]$; degree of $\mathcal{I}$
$I_{d_1, \dots, d_n}$	Iterated-Simplex polynomial (10)
$C_n^{(d)}$	Fuss–Catalan number (6)
$\mathcal{X}, \mathcal{X} $	variable block and its cardinality (Method 2)
ω	matrix multiplication exponent, $2 \leq \omega < 3$

2.1 Resultants

Resultants are classical tools in elimination theory used to determine whether a polynomial system has a common solution. We use the projection-operators viewpoint of [55].

Definition 1 (Resultant [55]). *Let $\mathcal{F} = \{f_1, \dots, f_n\} \subset \mathbb{K}[\mathbf{a}, \mathbf{x}]$. Set $\mathcal{I} = \langle \mathcal{F} \rangle$ and $J = \mathcal{I} \cap \mathbb{K}[\mathbf{a}]$. Elements of J are called projection operators: they vanish at parameter values for which $\mathcal{F}$ has a common solution over $\overline{\mathbb{K}}$. If J is principal, its generator (up to a nonzero scalar) is called the resultant of $\mathcal{F}$.*

The Sylvester resultant [31,82] for two univariate polynomials of degrees d_1, d_2 produces a matrix of size $d_1 + d_2$; the Bézout–Cayley resultant [67,31] reduces this to $\max(d_1, d_2)$. The Macaulay resultant [65] generalizes the Sylvester resultant to several multivariate polynomials but can produce very large matrices. The Dixon construction [33] is a multivariate extension of the Bézout-style approach: it preserves the compactness of the Bézout–Cayley matrix while extending it from two to n polynomials, often yielding much smaller matrices than Macaulay.

Remark 1. For $k > 1$, J need not be principal. Under suitable conditions, determinant-based methods compute a specific nonzero polynomial R that vanishes on the projection of the affine solution set, possibly with extraneous factors. In this paper, we refer to this determinant R as the resultant in a slightly informal sense, rather than as a projection polynomial.

2.2 Dixon Resultant

The Dixon resultant method [33,56] eliminates several variables simultaneously via a determinant. The procedure has four steps [56,51].

Step 1: Compute the Dixon polynomial. Introduce $n-1$ auxiliary variables $\bar{x}_1, \dots, \bar{x}_{n-1}$ and define the $n \times n$ cancellation matrix

$$C(\mathbf{x}, \bar{\mathbf{x}}) = \begin{bmatrix} f_1(x_1, x_2, \dots, x_{n-1}) \cdots f_n(x_1, x_2, \dots, x_{n-1}) \\ f_1(\bar{x}_1, x_2, \dots, x_{n-1}) \cdots f_n(\bar{x}_1, x_2, \dots, x_{n-1}) \\ f_1(\bar{x}_1, \bar{x}_2, \dots, x_{n-1}) \cdots f_n(\bar{x}_1, \bar{x}_2, \dots, x_{n-1}) \\ \vdots \quad \ddots \quad \vdots \\ f_1(\bar{x}_1, \bar{x}_2, \dots, \bar{x}_{n-1}) \cdots f_n(\bar{x}_1, \bar{x}_2, \dots, \bar{x}_{n-1}) \end{bmatrix}. \quad (1)$$

The Dixon polynomial is

$$\Delta_{\mathcal{F}}(\mathbf{x}, \bar{\mathbf{x}}) = \frac{\det(C(\mathbf{x}, \bar{\mathbf{x}}))}{\prod_{i=1}^{n-1} (x_i - \bar{x}_i)}. \quad (2)$$

Instead of performing an explicit symbolic division, we use the refined construction from [52]

$$\text{Row}(\hat{C}_1) = \text{Row}(C_1), \quad \text{Row}(\hat{C}_{i+1}) = \frac{\text{Row}(C_{i+1}) - \text{Row}(C_i)}{x_i - \bar{x}_i}, \quad (3)$$

for $i = 1, \dots, n-1$. Each numerator is divisible by $x_i - \bar{x}_i$ in $\mathbb{K}[\mathbf{x}, \bar{\mathbf{x}}, \mathbf{a}]$, so the quotient remains polynomial; the Dixon polynomial is $\Delta_{\mathcal{F}}(\mathbf{x}, \bar{\mathbf{x}}) = \det(\hat{C})$. This row-wise difference-quotient construction is used throughout the paper.

Step 2: Construct the Dixon matrix. The Dixon polynomial admits a bilinear form $\Delta_{\mathcal{F}}(\mathbf{x}, \bar{\mathbf{x}}) = X\mathcal{M}\bar{X}^T$, where X and $\bar{X}$ list monomials in $\mathbf{x}$ and $\bar{\mathbf{x}}$, respectively. The coefficient matrix $\mathcal{M}$ is the **Dixon matrix**: its rows are indexed by the $\mathbf{x}$ -monomials and its columns by the $\bar{\mathbf{x}}$ -monomials occurring in $\Delta_{\mathcal{F}}$, so it is in general rectangular and rank-deficient. Alternatively, $\mathcal{M}$ can be constructed directly via the recursive block method of Zhao and Fu [85], particularly relevant when few variables are eliminated and degrees are high [68].

Step 3: Extract a maximal-rank submatrix. In practice, the Dixon matrix $\mathcal{M}$ is often rectangular or rank-deficient, so its full determinant is unavailable or identically zero. Following the extension of Dixon elimination to singular matrices by KSY [56], we extract a maximal-rank square submatrix $\mathcal{M}'$ of size $\text{rank } \mathcal{M}$ whose determinant is nonzero. Following [52, 51], such a submatrix is typically identified efficiently after evaluating $\mathcal{M}$ at a generic point of $\mathbb{K}$.

Remark 2. The validity of using $\det(\mathcal{M}')$ relies on the KSY condition [56], also called the RSC criterion [3]: there exists a column whose monomial label is nonzero on the solution set under consideration and whose deletion lowers the rank over $\mathbb{K}(\mathbf{a})$. Failures of this condition appear rare with none observed in the cases tested in [56] or in this paper, and Albrecht [3, Section 4.2.3] addresses the nontermination issue in the RSC procedure through recursive specialization.

Step 4: Compute the Dixon resultant. Once $\mathcal{M}'$ is obtained, $\det(\mathcal{M}')$ is the Dixon resultant in $\mathbb{K}[\mathbf{a}]$. Under the KSY condition, it remains a valid projection polynomial for a singular Dixon matrix: it vanishes whenever the original system has a common solution in the eliminated variables. In general, $\det(\mathcal{M}')$ may contain extraneous factors.

Size of the Dixon matrix. The size of the Dixon matrix is a key parameter in the complexity analysis. A classical variable-wise upper bound [56, 68] is

$$B_{\text{var}} = (n-1)! \prod_{i=1}^{n-1} m_i, \quad (4)$$

where $m_i = \max_{1 \leq j \leq n} \deg_{x_i}(f_j)$ is the maximum degree in x_i over all polynomials in $\mathcal{F}$. Kapur and Saxena [54, Theorem 4.3] give a volume-based asymptotic estimate for multi-homogeneous systems. In the equal-total-degree setting, their volume expression is $(dn)^{n-1}/n!$. We use its integer part as the comparison bound

$$B_{\text{MH}} = \left\lfloor \frac{(dn)^{n-1}}{n!} \right\rfloor. \quad (5)$$

Section 3 derives exact generic support formulas for the total-degree setting and independently verifies the explicit inequality $D(\mathcal{F}) \leq B_{\text{MH}}$ from the Fuss–Catalan formula.

Influence of extraneous factors. The determinantal output may contain factors unrelated to the affine solution set.

Definition 2 (Extraneous factor). *Let Z be the Zariski closure of the projection of $V(\mathcal{I})$ onto the retained parameters. An irreducible factor e of the determinantal elimination polynomial is an **extraneous factor** if $V(e) \not\subseteq Z$.*

In the absence of such factors and of excess multiplicities, the total degree of the eliminant is bounded by $d_{\mathcal{I}}$, both when $k = 1$ and when $k \geq 2$; in either case it is bounded by the Bézout bound:

Theorem 1 (Bézout bound [46,37]). *Let $\mathcal{I} = \langle f_1, \dots, f_m \rangle \subset \mathbb{K}[x_1, \dots, x_N]$ with $\deg(f_i) = d_i$, and let $d_{\mathcal{I}}$ denote the degree of $\mathcal{I}$, i.e. the number of solutions counted with multiplicity when $\mathcal{I}$ is zero-dimensional, and the degree of the affine variety $V(\mathcal{I})$ in general. Then*

$$d_{\mathcal{I}} \leq \prod_{i=1}^m d_i.$$

In the presence of extraneous factors, the degree of the eliminant may exceed the Bézout bound of the underlying system, both inflating computational costs and introducing spurious components unrelated to the actual solution set.

2.3 Polynomial Matrix Determinant Computation

We summarize the determinant complexity models used in this paper; full details are provided in the full version [77].

Univariate determinants. For $P(t) \in \mathbb{K}[t]^{m \times m}$, the Hermite normal form (HNF) method of [60] has deterministic cost $T^{\text{HNF}} = \tilde{O}(m^\omega \max\{1, \lceil \delta \rceil\})$, where δ is the average column degree.

Multivariate determinants. For $P(\mathbf{y}) \in \mathbb{K}[y_1, \dots, y_v]^{m \times m}$, no single method is universally best: different algorithms can lead to substantially different complexity estimates depending on the matrix size, the number of variables, and the sparsity structure of the entries. We therefore consider several alternatives. Besides direct symbolic expansion by minors [48,38] and fraction-free Bareiss elimination [13], one may reduce to the univariate case via Kronecker substitution [63] and apply HNF, apply evaluation-interpolation on a tensor grid [48], or use sparse interpolation [49]. The corresponding time and space costs are summarized in Table 2.

Table 2. Determinant computation models used in this paper. Time counts field operations; space bounds count field elements in the workspace.

Method	Time	Space
Expansion [38,48]	$\tilde{O}(m2^m \mu U)$	$\tilde{O}\left(\binom{m}{\lfloor m/2 \rfloor} U\right)$
Bareiss [13,38]	$\tilde{O}(m^3 U^2)$	$\tilde{O}(m^2 U + U^2)$
Kronecker [63]	$\tilde{O}(m^\omega \delta)$	$\tilde{O}(m^\omega (\delta + 1))$
Interpolation [48]	$\tilde{O}(N(L + vB))$	$\tilde{O}(N + L)$
Sparse [49] ⁶	$\tilde{O}(LS + S \log q)$	$\tilde{O}(vS + L)$

In the table notation, b_i bounds $\deg_{y_i} \det(P)$ and fixes the Kronecker substitution base (chosen to avoid overflow in the encoded determinant); $B := \max_i b_i$; $N := \prod_{i=1}^v (b_i + 1)$ is the tensor-grid size used by interpolation; S bounds the number of nonzero terms of $\det(P)$; U bounds the term count of intermediate minors; L includes matrix evaluation and determinant computation (and, for sparse interpolation, simultaneous first derivatives); and $\mu \leq \binom{v+d}{d}$ bounds the dense monomial count of an entry of total degree at most d . All complexities are reported in field operations over $\mathbb{F}_q$; the derivations are given in the full version [77].

2.4 Security of AO Primitives

The sponge construction [20] builds hash functions from permutations. The relevant task for algebraic attacks is the constrained-input constrained-output [19] (CICO) problem.

Definition 3 (CICO problem). *For a permutation $F: \mathbb{F}_q^t \rightarrow \mathbb{F}_q^t$, the CICO- k problem is to find $x \in \mathbb{F}_q^{t-k} \times \{0\}^k$ and $y \in \mathbb{F}_q^{t-k} \times \{0\}^k$ with $F(x) = y$.*

Brute force requires $\sim q^k$ permutation calls; substantially faster algebraic solutions may translate into preimage or collision attacks in the sponge setting.

3 Improved Complexity Estimates for the Dixon Resultant

Newton-polytope bounds for unmixed systems [54] and multivariate support methods for mixed systems [27] provide the starting point for our analysis. We derive an explicit lattice-path bound from ordered total-degree bounds and prove that it is attained for generic systems in the corresponding coefficient space.

⁶ The sparse time bound is expected, with Monte Carlo correctness, and requires $\text{char}(\mathbb{F}_q) > B$ (Huang [49, Algorithm 9 and Theorem 12]); it therefore applies to large-prime fields but not to extension fields of the form $\mathbb{F}_{2^\ell}$.

Throughout this section, $D(\mathcal{F})$ is the size of the Dixon matrix $\mathcal{M}$ of $\mathcal{F}$ (Table 1); it upper-bounds $\rho = \text{rank } \mathcal{M}$ but is in general strictly larger (Remark 5). For generic systems, and under the characteristic hypothesis of Lemma 4, the two monomial counts coincide and $\mathcal{M}$ is square, which is the situation described by the formulas below. A system of n polynomials with $\deg_{\mathbf{x}}(f_i) \leq d_i$ is called a $(d_1, \dots, d_n)$ -system, and an $(n; d)$ -system when $d_1 = \dots = d_n = d$, following [54].

3.1 A Refined Bound on the Size of the Dixon Matrix

We bound the Dixon matrix size for general $(d_1, \dots, d_n)$ -systems via a lattice path count. The required background on polynomial supports, Newton polytopes, and lattice paths is reviewed in the full version [77].

Theorem 2 (Lattice Path Bound). *For a $(d_1, d_2, \dots, d_n)$ -system $\mathcal{F}$ with $d_1 \leq d_2 \leq \dots \leq d_n$, let*

$$a_i := \sum_{k=1}^i (d_{n+1-k} - 1), \quad i = 1, 2, \dots, n-1.$$

Then $D(\mathcal{F})$ is bounded above by the number of lattice paths from $(0, 0)$ to $(n-1, a_{n-1})$ such that for every $i = 1, 2, \dots, n-1$, the height y_i of the i -th horizontal step is at most a_i . Moreover, if $\text{char}(\mathbb{K})$ is zero or sufficiently large, this bound is attained for generic polynomial systems.

Remark 3. The bound and the generic matrix size are independent of d_1 . For nested total-degree supports, the support relations in [27, Theorems 3.1 and 5.1] imply that replacing the smallest-support equation by 1 preserves generic support. The proof below gives an explicit system attaining the bound.

This lattice path counting problem admits precise closed-form solutions through classical results in enumerative combinatorics.

Lemma 1. *Let $a = (a_1, a_2, \dots, a_{n-1})$ be an integer sequence with $0 \leq a_1 \leq a_2 \leq \dots \leq a_{n-1}$. Consider lattice paths from $(0, 0)$ to $(n-1, a_{n-1})$.*

(1) *The total number of lattice paths without any restriction equals*

$$\binom{(n-1) + a_{n-1}}{n-1}.$$

(2) *If $a_i = i(d-1)$ for some constant d and all $i = 1, \dots, n-1$, then the number of lattice paths such that the height y_i of the i -th horizontal step is at most a_i equals the Fuss–Catalan number with parameters (n, d)*

$$C_n^{(d)} = \frac{1}{(d-1)n+1} \binom{dn}{n}. \quad (6)$$

- (3) The number of lattice paths such that for every $i = 1, 2, \dots, n-1$ the height y_i of the i -th horizontal step is at most a_i is given by the determinant

$$\det_{1 \leq i, j \leq n-1} \left(\binom{a_i + 1}{j - i + 1} \right).$$

Proof. Parts (1) and (2) are classical results in enumerative combinatorics; see [47, 6]. Part (3) is a special case of Theorem 10.7.1 in [22] with $b_j = 0$ for all j . $\square$

By combining Theorem 2 and Lemma 1, we immediately obtain the following bounds.

Corollary 1. For a $(d_1, d_2, \dots, d_n)$ -system $\mathcal{F}$ with $d_1 \leq d_2 \leq \dots \leq d_n$, the following three bounds on the Dixon matrix size hold:

- (1) **Unrestricted Bound:**

$$D(\mathcal{F}) \leq \binom{\sum_{i=2}^n (d_i - 1) + n - 1}{n - 1} = \binom{\sum_{i=2}^n d_i}{n - 1}. \quad (7)$$

This bound is obtained by counting all lattice paths from $(0, 0)$ to $(n - 1, \sum_{i=2}^n (d_i - 1))$ without boundary constraints.

- (2) **Fuss-Catalan Bound (for $(n; d)$ -systems):** Assume $d_1 = \dots = d_n = d$.

$$D(\mathcal{F}) \leq C_n^{(d)} = \frac{1}{(d-1)n+1} \binom{dn}{n}. \quad (8)$$

The bound is attained for generic systems by Theorem 2 and Lemma 1(2), under the characteristic condition stated in Remark 5.

- (3) **Hessenberg Bound:**

$$D(\mathcal{F}) \leq \det_{1 \leq i, j \leq n-1} \left(\binom{\sum_{k=1}^i (d_{n+1-k} - 1) + 1}{j - i + 1} \right). \quad (9)$$

The bound is attained for generic systems by Theorem 2 and Lemma 1(3), under the characteristic condition stated in Remark 5.

Remark 4. The matrix in (9) is a Hessenberg matrix, hence its determinant can be computed in $\mathcal{O}(n^2)$ using specialized algorithms [26] or [66, Theorem 1.9].

Outline of proof of Theorem 2. We use the Iterated-Simplex Polynomial to connect the Dixon matrix size to lattice path counting. Lemma 2 describes its support by linear inequalities. Lemma 4 then bounds the Dixon matrix size by the monomial count of $I_{d_2-1, \dots, d_n-1}$. Finally, Lemma 5 counts these monomials via a bijection with lattice paths below a piecewise linear boundary.

Definition 4 (Iterated-Simplex Polynomial). Let n be a positive integer and $(d_1, d_2, \dots, d_n)$ be a sequence of nonnegative integers. The Iterated-Simplex polynomial is defined as

$$I_{d_1, d_2, \dots, d_n}(x_1, \dots, x_n) := \prod_{k=1}^n \left(\sum_{\alpha \in \mathbb{Z}_{\geq 0}^k, \alpha_1 + \dots + \alpha_k \leq d_k} x_1^{\alpha_1} \dots x_k^{\alpha_k} \right), \quad (10)$$

where the k -th factor represents the complete sum of monomials in the first k variables with total degree at most d_k .

Now we characterize the support of the Iterated-Simplex polynomial in terms of linear inequalities.

Lemma 2. The support of the Iterated-Simplex Polynomial $I_{d_1, \dots, d_n}$, taken over $\mathbb{Z}$, is given by

$$\text{Supp}(I_{d_1, \dots, d_n}) = \left\{ \alpha \in \mathbb{Z}_{\geq 0}^n : \sum_{l=n-j+1}^n \alpha_l \leq \sum_{i=1}^j d_{n-i+1} \text{ for } j = 1, \dots, n \right\}. \quad (11)$$

A proof is given in the full version [77].

To compare different degree sequences, we establish a monotonicity property with respect to the trailing partial sums; it covers both reorderings and entry-wise decreases of the degrees.

Lemma 3. Let $(d_1, \dots, d_n)$ and $(d'_1, \dots, d'_n)$ be sequences of nonnegative integers satisfying $\sum_{j=1}^k d'_{n-j+1} \leq \sum_{j=1}^k d_{n-j+1}$ for every $k = 1, \dots, n$. Then

$$\text{Supp}(I_{d'_1, \dots, d'_n}) \subseteq \text{Supp}(I_{d_1, \dots, d_n}).$$

In particular, if $d_1 \leq \dots \leq d_n$, the hypothesis holds for any permutation of a sequence obtained by decreasing some of the d_i .

Proof. Let $\alpha \in \text{Supp}(I_{d'_1, \dots, d'_n})$. By Lemma 2, for every $k = 1, \dots, n$,

$$\sum_{\ell=n-k+1}^n \alpha_\ell \leq \sum_{j=1}^k d'_{n-j+1} \leq \sum_{j=1}^k d_{n-j+1}.$$

These are precisely the inequalities defining $\text{Supp}(I_{d_1, \dots, d_n})$, so the inclusion follows. If the degrees are first decreased and then reordered, any k resulting entries sum to at most the k largest original entries. Under the sorted ordering, their sum is $d_{n-k+1} + \dots + d_n$, which verifies the hypothesis. $\square$

Connection to the Dixon Matrix Size. We now connect the Dixon matrix size to the Iterated-Simplex Polynomial.

Lemma 4. *Let $\mathcal{F} = \{f_1, \dots, f_n\} \subset \mathbb{K}[x_1, \dots, x_{n-1}]$ have total degrees at most $d_1 \leq \dots \leq d_n$. Then the Dixon matrix size is bounded above by the number of monomials in the Iterated-Simplex Polynomial $I_{d_2-1, \dots, d_n-1}$, taken over $\mathbb{Z}$. This bound is attained for generic systems if $\text{char}(\mathbb{K})$ is zero or sufficiently large.*

Proof. Let $S = \text{Supp}(I_{d_2-1, \dots, d_n-1})$ over $\mathbb{Z}$, and write $N^* = |S|$. In the bilinear form $\Delta_{\mathcal{F}} = X \mathcal{M} \bar{X}^T$, the column and row labels are the monomials in $\bar{x}$ and x , respectively.

Upper bound. We use the support argument of [54, Lemma 3.3 and Theorem 3.4] with the refined matrix (3). Its first row has no dual variables. For $i \geq 2$, the entry $\hat{C}_{i,j}$ involves only $\bar{x}_1, \dots, \bar{x}_{i-1}$ and has dual degree at most $d_j - 1$, since a difference quotient lowers the degree by one. Write $\text{Supp}_{\bar{x}}$ for the set of dual-variable exponent vectors, treating x as coefficient variables. For each permutation $\sigma \in S_n$, the corresponding determinant term satisfies

$$\text{Supp}_{\bar{x}} \left(\prod_{i=1}^n \hat{C}_{i, \sigma(i)} \right) \subseteq \text{Supp}(I_{d_{\sigma(2)}-1, \dots, d_{\sigma(n)}-1}) \subseteq S.$$

Indeed, in the main-diagonal term the last $n-1$ rows have the sorted degree bounds $d_2 - 1, \dots, d_n - 1$. For any other permutation σ , a monomial with dual exponent vector α satisfies, for $1 \leq k < n$,

$$\sum_{\ell=n-k}^{n-1} \alpha_{\ell} \leq \sum_{i=n-k+1}^n (d_{\sigma(i)} - 1) \leq \sum_{j=n-k+1}^n (d_j - 1).$$

The second inequality bounds the selected degrees by the k largest ones. Lemma 3 therefore places every determinant term in S . Summing these terms can remove monomials through cancellation, but cannot introduce monomials outside S . Hence $\Delta_{\mathcal{F}}$ has at most N^* dual monomials.

For the row count, use the variable-reversal symmetry of [27, proof of Proposition 5.1]. Let $\mathcal{F}^{\text{rev}}$ replace each x_i by x_{n-i} , and let τ interchange x_i with $\bar{x}_{n-i}$. Reversing the rows of the cancellation matrix gives

$$\tau(\Delta_{\mathcal{F}}) = \pm \Delta_{\mathcal{F}^{\text{rev}}}. \quad (12)$$

Since $\mathcal{F}^{\text{rev}}$ has the same degree bounds, its dual-monomial bound gives the required N^* bound for the original monomials of $\Delta_{\mathcal{F}}$.

Attaining the bound. Consider the nested system

$$f_1^* = 1, \quad f_i^* = (x_1 + \dots + x_{i-1})^{d_i}, \quad 2 \leq i \leq n.$$

Taking $f_1^* = 1$ is allowed because the prescribed degrees are upper bounds. For $j < i$, the polynomial f_j^* does not involve x_{i-1} . Consequently, the two evaluations

in the numerator of the row- i difference quotient agree, giving $\widehat{C}_{i,j} = 0$. Thus

$$\widehat{C} = \begin{bmatrix} 1 & f_2^* & \cdots & f_n^* \\ 0 & \widehat{C}_{2,2} & \cdots & \widehat{C}_{2,n} \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & \widehat{C}_{n,n} \end{bmatrix}, \quad \Delta_{\mathcal{F}^*} = \prod_{i=2}^n \widehat{C}_{i,i}.$$

Put $s_j = \bar{x}_1 + \cdots + \bar{x}_j$, with $s_0 = 0$. The diagonal entries are explicit difference quotients:

$$\begin{aligned} \widehat{C}_{i,i} &= \frac{s_{i-1}^{d_i} - (s_{i-2} + x_{i-1})^{d_i}}{x_{i-1} - \bar{x}_{i-1}} \\ &= - \sum_{t=0}^{d_i-1} (s_{i-2} + x_{i-1})^t s_{i-1}^{d_i-1-t}. \end{aligned}$$

Here the second equality is the difference-of-powers identity. Setting all $x_i = 1$ gives

$$\Delta_{\mathcal{F}^*}(\mathbf{1}, \bar{\mathbf{x}}) = (-1)^{n-1} \prod_{i=2}^n \left(\sum_{t=0}^{d_i-1} (1 + s_{i-2})^t s_{i-1}^{d_i-1-t} \right).$$

Each factor has degree at most $d_i - 1$ and involves only the first $i - 1$ dual variables. Conversely, fix a monomial of degree $q \leq d_i - 1$ in these variables. Taking $t = d_i - 1 - q$ and the constant term of $(1 + s_{i-2})^t$ leaves s_{i-1}^q , whose multinomial expansion contains that monomial. The factor therefore has exactly the complete simplex support of degree $d_i - 1$. All its coefficients are positive integers, so their products cannot cancel in characteristic zero or sufficiently large characteristic (Remark 5). Multiplying the factors gives precisely the support S of $I_{d_2-1, \dots, d_n-1}$. Since specializing $\mathbf{x}$ can only remove dual monomials, the unspecialized Dixon polynomial also attains the dual-monomial bound.

Generic attainment. Each coefficient just exhibited in $\Delta_{\mathcal{F}}(\mathbf{1}, \bar{\mathbf{x}})$ is a polynomial in the input coefficients, nonzero at $\mathcal{F}^*$. The coefficient space over $\overline{\mathbb{K}}$ is an affine space and hence irreducible. The simultaneous nonvanishing of these finitely many coefficient polynomials defines a nonempty open set U , which is therefore dense; see [31, Chapter 3, §5, Definition (5.6), p. 115]. Reversing the variable order permutes the input coefficients, so $U^{\text{rev}} := \{\mathcal{F} : \mathcal{F}^{\text{rev}} \in U\}$ is also dense and open. On $U \cap U^{\text{rev}}$, equation (12) ensures that $\Delta_{\mathcal{F}}$ has exactly N^* distinct monomials in each of $\mathbf{x}$ and $\bar{\mathbf{x}}$. Thus the matrix is generically square of size N^* . $\square$

Remark 5. The size $D(\mathcal{F})$ counts row and column monomials and upper-bounds $\rho = \text{rank } \mathcal{M}$; generic attainment of this size does not assert full rank. The upper bound holds over any field. A sufficient characteristic condition for attainment is

$$\text{char}(\mathbb{K}) = 0 \quad \text{or} \quad \text{char}(\mathbb{K}) > \prod_{i=2}^n d_i(i-1)^{d_i-1},$$

because the product on the right is the sum of the absolute values of the integer coefficients in the specialized product above, and hence bounds each coefficient. In small characteristic, cancellations may shrink the support: for instance, over $\mathbb{F}_2$, $(1+x_1)(1+x_1+x_2) = 1+x_2+x_1^2+x_1x_2$, so the monomial x_1 disappears.

From Polyhedral Constraints to Lattice Paths. We now establish a bijection between lattice points satisfying the polyhedral constraints and lattice paths below a general boundary.

Lemma 5. *Let $a_1 \leq a_2 \leq \dots \leq a_n$ be nonnegative integers. The number of lattice points $(\alpha_1, \dots, \alpha_n) \in \mathbb{Z}_{\geq 0}^n$ satisfying*

$$\alpha_1 + \dots + \alpha_k \leq a_k, \quad k = 1, \dots, n,$$

is equal to the number of lattice paths from $(0, 0)$ to (n, a_n) such that for every $i = 1, 2, \dots, n$ the height y_i of the i -th horizontal step is at most a_i .

Proof. Given a lattice point, set $\alpha_{n+1} = a_n - \sum_{i=1}^n \alpha_i$ and form the path

$$U^{\alpha_1} R U^{\alpha_2} R \dots U^{\alpha_n} R U^{\alpha_{n+1}},$$

where U and R denote upward and rightward steps. This path has exactly n horizontal steps, and its k -th horizontal step has height $\alpha_1 + \dots + \alpha_k \leq a_k$. The final α_{n+1} upward steps bring it to (n, a_n) .

Conversely, given an admissible path, let α_i count the upward steps after the $(i-1)$ -st horizontal step and before the i -th, taking the starting point when $i = 1$. The height of the k -th horizontal step is then $\alpha_1 + \dots + \alpha_k$, so the required inequalities hold. The endpoint uniquely fixes the remaining upward steps. The two constructions are inverse to each other, giving the desired bijection. $\square$

Proof of Theorem 2. By Lemma 4, it remains to count the monomials of $I_{d_2-1, \dots, d_n-1}$. Applying Lemma 2 in $n-1$ variables, their exponent vectors satisfy

$$\sum_{\ell=n-j}^{n-1} \alpha_\ell \leq \sum_{i=1}^j (d_{n-i+1} - 1), \quad 1 \leq j < n.$$

Reversing the coordinates via $\beta_i = \alpha_{n-i}$ turns these inequalities into the prefix constraints $\beta_1 + \dots + \beta_j \leq a_j$. By Lemma 5, the resulting nonnegative lattice points correspond exactly to paths from $(0, 0)$ to $(n-1, a_{n-1})$ whose j -th horizontal step has height at most a_j . This is precisely the boundary stated in the theorem. Generic attainment follows from Lemma 4. $\square$

Comparison with Existing Bounds Our bound refines the equal-degree volume estimate [54] and extends the dense bivariate [28] and uniform quadratic [79] cases to arbitrary dimensions and unequal degree bounds. For $n = 3$ and equal degrees, our formula gives $C_3^{(d)} = d(3d-1)/2$, which also follows from [28,

Theorem 4.4]. For $d = 2$, it recovers the Catalan count of [79]. For general n , comparison with the volume estimate of [54, Theorem 4.3] gives

$$C_n^{(d)} = \frac{1}{n!} \prod_{j=0}^{n-2} (nd - j) \leq \frac{(nd)^{n-1}}{n!},$$

which also verifies the integer volume bound B_{MH} in (5). Numerical comparisons and support-size experiments are reported in the full version [77].

3.2 Complexity Estimates via Our Refined Bound

For clarity of exposition, we focus on $(n; d)$ -systems with $n \geq 2$, $d \geq 2$, and $k \geq 1$, where all polynomials have total degree d in all variables. Consider $\mathcal{F} = \{f_1, \dots, f_n\} \subset \mathbb{K}[\mathbf{x}, \mathbf{a}]$ consisting of n equations in $n-1+k$ variables, where $\mathbf{x} = (x_1, \dots, x_{n-1})$ are the eliminated variables and $\mathbf{a}$ the retained parameters. By Corollary 1(2), the generic Dixon matrix size is

$$C_n^{(d)} = \frac{1}{n(d-1)+1} \binom{nd}{n}.$$

Since $\rho \leq D(\mathcal{F})$, the quantity $C_n^{(d)}$ also upper-bounds the size of the extracted submatrix $\mathcal{M}'$, so the estimates below are upper bounds on the actual cost.

We give a stage-by-stage outline of the relevant model choices below; here we consider the determinant methods with the lowest theoretical complexity or the fastest experimental performance. Detailed parameter derivations, complexity formulas for all five determinant models, and the full Step 1 vs Step 4 comparison are available in the full version [77].

Step 1: Compute the cancellation determinant. This step computes the determinant of the $n \times n$ refined cancellation matrix (3), whose entries are polynomials in $\ell = 2n+k-2$ variables (the $n-1$ original variables, $n-1$ dual variables, and k parameters), with total parameter-degree bounded by $B_1 := nd$. This is the *many-variable, small-matrix* regime, where expansion by minors and sparse interpolation are most competitive. The abstract parameters governing these models admit the following explicit expressions in $(n, d, k, C_n^{(d)})$; their derivation appears in the full version [77]:

$$\mu_1 := \binom{n-1+k+d}{n-1+k}, S_1 := (C_n^{(d)})^2 \binom{nd+k}{k}, L_1 = \tilde{O}\left(n^2 \binom{n-1+k+d}{n-1+k}\right), \quad (13)$$

where μ_1 counts the monomials of one dense polynomial of total degree d in $n-1+k$ variables, the factor $(C_n^{(d)})^2$ in S_1 comes from the Dixon bilinear support and $\binom{nd+k}{k}$ from the parameter monomials of degree at most B_1 , and the constant-determinant cost inside L_1 is absorbed into the matrix-evaluation term. For these parameters, S_1 also bounds every intermediate minor, and the

determinant admits a division-free straight-line program of length $O(n^2\mu_1)$; see the full version [77]. Substituting (13) into the two preferred determinant models yields

$$T_1^{\text{minor}} = \tilde{O}(n2^n\mu_1S_1) = \tilde{O}\left(n2^n\binom{n-1+k+d}{n-1+k}(C_n^{(d)})^2\binom{nd+k}{k}\right), \quad (14)$$

$$T_1^{\text{sparse}} = \tilde{O}(L_1S_1 + S_1\log q) = \tilde{O}\left(n^2\binom{n-1+k+d}{n-1+k}(C_n^{(d)})^2\binom{nd+k}{k}\right) \quad (15)$$

(the term $S_1\log q$ is dominated by L_1S_1 up to polylogarithmic factors). The sparse model saves a factor $2^n/n$ over expansion, but in all our experiments cached expansion by minors is faster, its constant factors being much smaller.

Step 2: Construct the Dixon matrix. Extracting the Dixon matrix coefficients from the cancellation determinant is dominated by Step 1. The block recursion of Zhao and Fu [85] builds $\mathcal{M}$ directly; following Qin et al. [68], the resulting surrogate T_2^{FDixon} is attractive when few variables are eliminated and degrees are high, but its $(n!)^3$ factor (derived in the full version [77]) makes it less competitive otherwise.

Step 3: Extract a maximal-rank submatrix. After evaluating the entries (a cost absorbed by Step 1), Step 3 extracts a maximal-rank submatrix numerically, with linear-algebra cost

$$T_3 = \mathcal{O}\left((C_n^{(d)})^\omega\right). \quad (16)$$

Given the entry degrees, the degree-aware weights of Section 4.1 are computed by a single pass over the $(C_n^{(d)})^2$ entries, dominated by (16).

Step 4: Compute the Dixon resultant. This step computes the symbolic determinant of $\mathcal{M}'$, a polynomial matrix of size at most $C_n^{(d)}$ in the k retained parameters $\mathbf{a}$. In the Kronecker+HNF model, the complexity is governed by the entry-wise parameter degree $E_4 \leq nd$, while in the evaluation-interpolation model it is governed by the total degree D_4 of the elimination polynomial. Under Heuristic 2, the elimination polynomial satisfies the Bézout degree bound $D_4 \leq d^n$. Put $C = C_n^{(d)}$ and $\mu_4 = \binom{nd+k}{k}$. Including matrix evaluation, the corresponding parameters are

$$N_4 \leq (d^n + 1)^k, \quad L_4 = \tilde{O}(C^\omega + C^2\mu_4), \quad \delta_4 = nd(Cnd + 1)^{k-1}. \quad (17)$$

where δ_4 bounds the average column degree after Kronecker substitution and reduces to $\delta_4 = nd$ when $k = 1$. Therefore,

$$T_4^{\text{HNF}} = \tilde{O}\left((C_n^{(d)})^\omega \delta_4\right) = \tilde{O}\left((C_n^{(d)})^\omega nd(C_n^{(d)} \cdot nd + 1)^{k-1}\right), \quad (18)$$

$$T_4^{\text{eval}} = \tilde{O}(N_4(L_4 + kD_4)) = \tilde{O}((d^n + 1)^k (C^\omega + C^2\mu_4 + kd^n)). \quad (19)$$

Overall comparison. Let T_1^{best} and T_4^{best} be the minimum cost across all considered models for Steps 1 and 4, respectively. Then $\mathcal{T}_{\text{Dixon}} = \tilde{O}(\max\{T_1^{\text{best}}, T_4^{\text{best}}\})$. At the idealized boundary $\omega = 2$, both stages must be retained. For $\omega > 2$ and fixed k , under the applicable field and degree hypotheses, Step 4 is asymptotically dominant among the displayed cost bounds in the fixed- d or fixed- n regimes analyzed in the full version [77]. It also dominates the running time in our experiments, so we use T_4^{best} as the asymptotic estimate.

Well-determined case. For $k = 1$, the final determinant is a $C_n^{(d)} \times C_n^{(d)}$ univariate polynomial matrix for which the HNF algorithm of [60] is typically optimal:

$$T_4^{\text{HNF}} = \tilde{O}\left((C_n^{(d)})^\omega nd\right) = \tilde{O}\left(nd \left(\frac{1}{n(d-1)+1} \binom{nd}{n}\right)^\omega\right). \quad (20)$$

For Step 1, the most competitive models are sparse interpolation and cached expansion by minors; substituting $k = 1$ into (14) and (15) gives

$$T_1^{\text{minor}} = \tilde{O}\left(n^2 d 2^n \binom{n+d}{n} (C_n^{(d)})^2\right), \quad (21)$$

$$T_1^{\text{sparse}} = \tilde{O}\left(n^3 d \binom{n+d}{n} (C_n^{(d)})^2\right). \quad (22)$$

With the best applicable construction model, Step 4 is asymptotically dominant for $\omega > 2$ in the regimes analyzed in the full version [77], and is also the empirical bottleneck. We therefore adopt $T = T_4^{\text{HNF}}$ as the asymptotic estimate for well-determined systems; the same reference gives the model-specific comparisons.

Remark 6. For mixed-degree systems, we bound $D(\mathcal{F})$ using the Hessenberg Bound of Corollary 1(3) and take the maximum cost over all stages. In particular, for $k = 1$ and total-degree bounds $d_1, \dots, d_n$, Step 4 costs $\tilde{O}((\sum_i d_i) D(\mathcal{F})^\omega)$ and dominates in most cases. We omit a detailed mixed-degree analysis.

3.3 Comparison with Gröbner Basis Methods

Gröbner basis methods solve polynomial systems by computing a grevlex order basis via F4/F5 [10,35] and then converting to lexicographic order via FGLM [36]. We compare Dixon with these methods on $(n; d)$ -systems.

Well-Determined Systems ($m = n$) Numerous studies [10,11,16,73,58] have established theoretical complexity bounds for Gröbner basis algorithms. Table 3 summarizes these bounds and their ratios relative to Dixon.

The Gröbner-based approach includes both the grevlex basis computation and the change to lexicographic order, so the relevant comparison is with the

Table 3. Arithmetic cost bounds for well-determined $(n; d)$ -systems, with $d_{\text{reg}} = n(d-1) + 1$, $d_{\mathcal{I}} = d^n$, and $t = \Theta(d^{n-1}/\sqrt{n})$. Ratios omit logarithmic factors. The three displayed Gröbner-basis ratios (classical, F5, and rectangular) have the same orders in both standard asymptotic directions. The displayed FGLM and BNS ratios use n fixed and $d \rightarrow \infty$. Complete derivations are given in the full version [77].

Method	Complexity	Ratio to Dixon
Dixon Resultant	$\tilde{\mathcal{O}}\left(nd \cdot \left(\frac{1}{n(d-1)+1} \binom{nd}{n}\right)^\omega\right)$	1
GB (classical) [10]	$\mathcal{O}\left(\binom{n+d_{\text{reg}}}{d_{\text{reg}}}\right)^\omega$	$\mathcal{O}((nd)^{\omega-1})$
GB (F5) [11,16]	$\mathcal{O}\left(n \cdot d_{\text{reg}} \cdot \binom{n+d_{\text{reg}}-1}{d_{\text{reg}}}\right)^\omega$	$\mathcal{O}(n^{\omega+1})$
GB (rectangular) [11,75]	$\mathcal{O}\left(n \cdot \binom{n+d_{\text{reg}}-d}{n} \cdot \binom{n+d_{\text{reg}}}{n}^{\omega-1}\right)$	$\mathcal{O}(n^\omega d^{\omega-1})$
GB (per-deg) [73,58]	$\mathcal{O}\left(n \cdot \sum_{i=0}^{d_{\text{reg}}} \binom{n+i-d-1}{i-d} \binom{n+i-1}{i}^{\omega-1}\right)$	—
FGLM [36,58]	$\mathcal{O}(n \cdot d_{\mathcal{I}}^\omega)$	$\mathcal{O}(n^{3\omega/2} d^{\omega-1} e^{-n\omega})$
BNS order change [18]	$\tilde{\mathcal{O}}(t^{\omega-1} d_{\mathcal{I}})$	$\mathcal{O}(n^{\omega-1/2} e^{-n\omega})$

combined F4/F5+FGLM cost. We also include the HNF-based change-of-order algorithm of [18] as an alternative to FGLM. In both asymptotic regimes considered, the displayed Dixon bound is asymptotically smaller than the combined bound. However, these Gröbner basis costs are upper bounds that do not fully capture practical F4/F5 optimizations. Finer estimates for F5 [11] and F4 Tracer [59] are available under additional assumptions and not included in our comparison. The ratios therefore compare general upper bounds, while Section 4.2 provides practical comparisons. Figure 1 evaluates the unsimplified cost expressions as n varies, including the per-degree sum; further plots and ratio derivations are given in the full version [77].

Overdetermined Systems ($m > n$) Gröbner basis methods are typically preferable here: the degree of regularity is significantly lower than for well-determined systems [10,12], and the grevlex basis often yields univariate polynomials directly, eliminating the FGLM step. The Dixon resultant, in contrast, needs exactly n equations to eliminate $n-1$ variables and cannot leverage redundancy: one selects n equations and verifies the rest afterward, at slightly higher cost than the well-determined case.

Underdetermined Systems ($m < n$) For complete solving, a common crypt-analytic strategy is to guess $n-m$ variables and solve the resulting subsystem

Complexity Comparison: Dixon vs Gröbner Basis Methods
($d=2$, $\omega=2.81$)

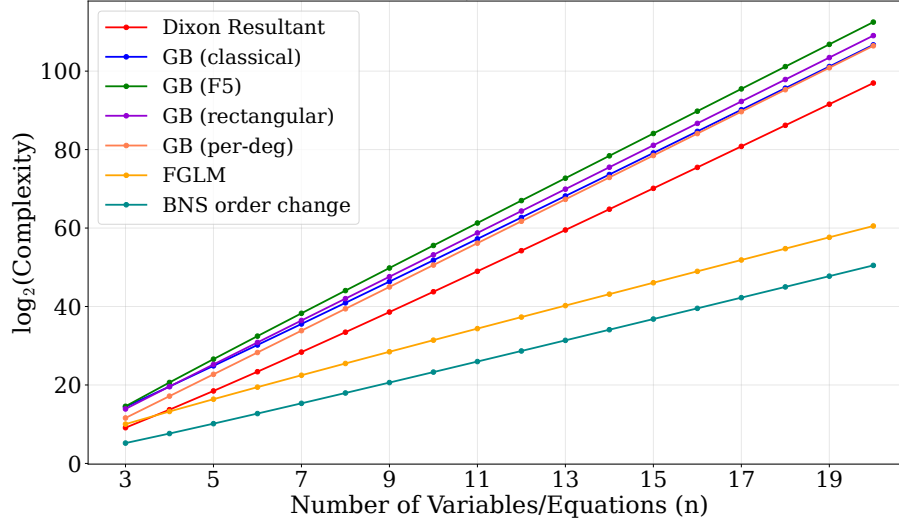


Fig. 1. Complexity comparison varying n with $d = 2$, $\omega = 2.81$.

using Gröbner bases; we do not propose Dixon as a replacement. For elimination in this positive-dimensional setting, however, Dixon treats the retained variables as parameters and eliminates several variables at once, producing explicit relations that can be combined with guessing or meet-in-the-middle decompositions; by contrast, Gröbner-basis conversion in the same setting may use Gröbner walks [29], whose cost can be difficult to predict.

In summary, the two methods are complementary: Dixon and Gröbner basis are comparable on well-determined systems; Gröbner basis is typically preferable for overdetermined systems; Dixon is attractive for underdetermined systems as a determinant-based elimination method.

4 Optimized Implementation of the Dixon Resultant

While the theoretical complexity analysis highlights the advantages of Dixon resultant methods, bridging the gap to practical efficiency requires overcoming several significant hurdles. Currently, off-the-shelf computer algebra systems offer limited support: Dixon resultants are not natively implemented in advanced solvers such as Magma [23], and although implementations exist in Fermat [62] and through Jinadu–Monagan’s packages in Maple [52,51], their performance is often unsatisfactory at scale. Beyond software limitations, extraneous factors must be carefully controlled or the classical Bézout bound becomes inapplicable, and large-scale polynomial-matrix operations must be handled efficiently. This section addresses these issues.

4.1 Reducing Extraneous Factors via Degree-Aware Submatrix Selection

The primary challenge in implementing the Dixon resultant arises from the *extraneous factors* in Definition 2, which do not correspond to actual solutions of the original system and can substantially inflate the resultant degree beyond the Bézout bound (Theorem 1). Without extraneous factors, the degree is controlled by the ideal degree and hence by Bézout, providing both theoretical elegance and computational efficiency.

Chitcheba and Kapur reduce extraneous degree through support translations and dialytic multiplier selection [28, Section 9]. Qin et al. [69] proposed a method based on parameter specialization and regular-chain computation; however, the regular-chain step incurs additional overhead. Our strategy selects low-degree maximal-rank submatrices of the original Dixon matrix. Our experimental results show that the proposed greedy selection algorithm reduces redundant factors, and in most cases the resulting Dixon determinants remain at or below the Bézout bound.

Degree-aware submatrix selection strategy. Following the determinant degree estimation framework of Labahn et al. [60], for any nonsingular polynomial matrix A ,

$$\deg(\det(A)) \leq \min(|\text{rdeg}(A)|, |\text{cdeg}(A)|),$$

where $|\text{rdeg}(A)|$ and $|\text{cdeg}(A)|$ denote the sum of row degrees and column degrees, respectively. Our method is based on the following working hypothesis:

Heuristic 1 (Degree Minimization Heuristic) *For two maximal-rank square submatrices A and B of the same Dixon matrix, if*

$$\min(|\text{rdeg}(A)|, |\text{cdeg}(A)|) > \min(|\text{rdeg}(B)|, |\text{cdeg}(B)|),$$

then we heuristically expect B to have a lower determinant degree.

The intuition is that a smaller row/column-sum degree bound tends to indicate a smaller determinant degree, motivating our greedy strategy to favor low-degree maximal-rank submatrices and thereby potentially reduce extraneous factors.

Remark 7. Labahn et al. [60] also used a refined bound $\deg(\det(A)) \leq \max_{\pi \in S_n} \sum_i \deg(a_{i, \pi(i)})$, where S_n is the symmetric group on $\{1, \dots, n\}$. For a given matrix, this bound can be computed in $\mathcal{O}(n^3)$ by maximum-weight bipartite matching. We use row/column weights computable in $\mathcal{O}(n^2)$ as a cheaper but coarser guide to submatrix selection.

Degree-aware selection algorithm. Based on Heuristic 1, we design a greedy algorithm that combines rank verification with degree-aware ordering of rows and columns. The core idea is to assign degree weights to rows and columns of the Dixon matrix and prioritize those with lower degrees during Gaussian elimination, preserving the rank at the evaluation point while heuristically favoring a low determinant degree. Full pseudocode is provided in the full version [77].

If each coordinate is sampled independently and uniformly from a finite set $A \subseteq \mathbb{K}$, the probability of rank loss is at most $\min\{1, \deg(h)/|A|\}$ for any fixed nonzero maximal-rank minor h , by the Schwartz–Zippel lemma [72,86]. See also [53,51].

Heuristic 2 *Let $\mathcal{P} = \{p_1, \dots, p_n\}$ be a generic polynomial system with degrees $d_1, \dots, d_n$, and $\mathcal{M}$ its Dixon matrix. The submatrix selected by the degree-aware strategy above yields a resultant R satisfying $\deg(R) \leq \prod_{i=1}^n d_i$.*

This is supported by computational experiments reported in the full version [77], with rare observed violations of the Bézout bound by only 1–2 degrees. Passing this test does not establish the absence of extraneous factors or excess multiplicities. The estimates using $D_4 \leq \prod_i d_i$ assume Heuristic 2; the stronger bound $D_4 \leq d_{\mathcal{I}}$ additionally assumes their absence.

4.2 Implementation

We developed a comprehensive, open-source C implementation of the Dixon resultant method, `DRSolve`, built upon the high-performance `FLINT` [80] and `PML` [81,50] libraries, available at <https://github.com/drsolve/drsolve>. It supports operations over prime fields $\mathbb{F}_p$ of arbitrary size, general finite fields $\mathbb{F}_q$, and $\mathbb{Q}$. It incorporates most Dixon construction techniques from [61,85,51] and most polynomial-matrix determinant techniques from [60,49,48,63]. It provides:

- **Elimination and solving:** Eliminate $n - 1$ variables from n polynomials, and compute all solutions of zero-dimensional systems.
- **Flexible determinant algorithms:** Support for all determinant computation methods discussed in this paper.
- **Optimized arithmetic:** Binary extension fields $\mathbb{F}_{2^8}, \mathbb{F}_{2^{16}}, \mathbb{F}_{2^{32}}, \mathbb{F}_{2^{64}}, \mathbb{F}_{2^{128}}$, and multi-modular techniques with CRT-based reconstruction over $\mathbb{Q}$ and large prime fields.
- **Complexity estimation:** A tool for estimating the cost of a polynomial system, or of a parameter set such as the number of variables and degrees.

Experimental results. As discussed in Section 3.3, Dixon resultants are well suited to parametric elimination, while Gröbner bases are particularly effective for overdetermined solving. We therefore focus our benchmarks on well-determined systems, a common setting well suited to both methods.

All benchmarks were performed on a dual-socket AMD EPYC 7302 server. We compared our Dixon implementation with the Gröbner basis solvers in

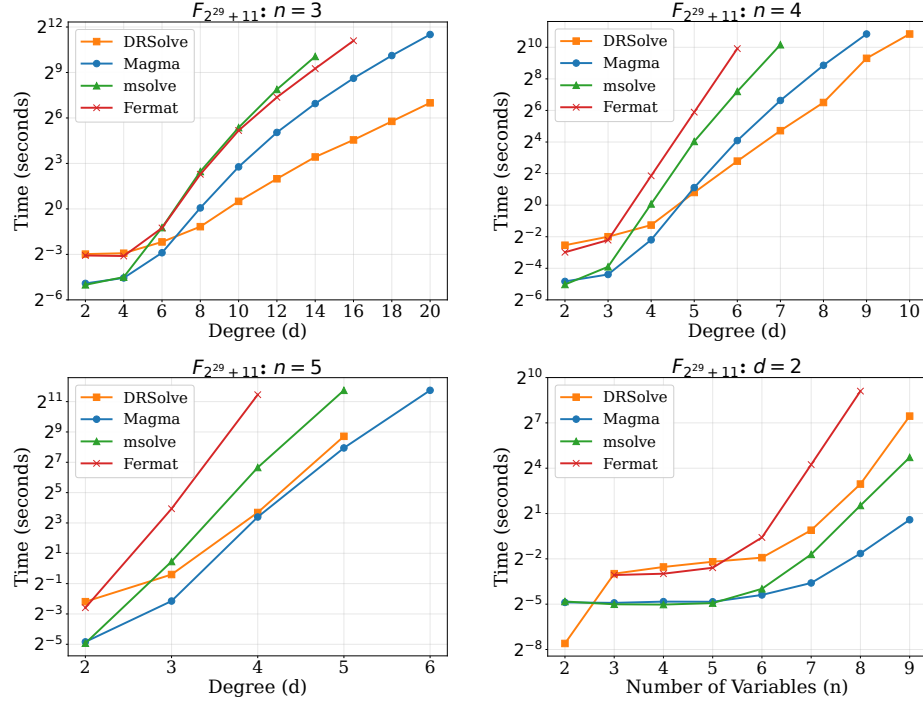


Fig. 2. Performance comparison (time in seconds) of DRSolve 0.3.0, Magma 2.29-7, msolve 0.9.5, and Fermat 7.9b on randomly generated dense polynomial systems over $\mathbb{F}_p$ with $p = 2^{29} + 11$. The four panels show degree sweeps for $n = 3, 4, 5$ and a variable-count sweep for $d = 2$; here n denotes the number of variables (equal to the number of equations). A curve stops where the corresponding solver exceeded one hour.

Magma [23]/msolve [17], and with the high-performance Dixon implementation Fermat [62]. Jinadu–Monagan’s Maple implementation is designed for $\mathbb{Q}$ and is not included in our finite-field benchmarks. Our advantage over Fermat is primarily due to the efficient polynomial-matrix routines in FLINT/PML, in particular the HNF algorithm; for Fermat, we used interpolation, which was the fastest option in our tests.

The comparison between Dixon implementations and Gröbner basis solvers reveals a clear trade-off: Magma performs best for systems with more variables and lower degrees, possibly benefiting from F4/F5 optimizations, while DRSolve performs best for systems with fewer variables and higher degrees.

5 Application to AO Primitives

This section organizes the applications around three elimination patterns. The goal is to highlight the solving methodology first and to use concrete AO primitives as representative case studies. We focus on solving strategies rather than

on new algebraic models for AO primitives, reusing the best-known models from the literature and reevaluating them through the Dixon-resultant viewpoint.

Remark 8. Although much of the preceding discussion is developed under genericity assumptions, the structural bounds remain applicable to non-generic polynomial systems arising in AO primitives. Estimates relying on heuristic assumptions remain conditional. These bounds may be loose for structured instances, but still provide useful complexity estimates for algebraic analysis.

5.1 Method 1: Direct elimination

Method 1 applies the Dixon resultant directly to a square polynomial system. Starting from n equations in n variables, we eliminate $n - 1$ variables, solve the resulting elimination polynomial in the remaining parameter, and then back-substitute recursively. We estimate the complexity based on the analysis in Section 3.2. This is the classical Dixon solving workflow of [79,3], instantiated here with the bounds of Section 3 and the implementation of Section 4.

This method is best suited to Type I (low-degree) primitives, whose CICO systems admit a dense but uniformly low-degree algebraic description, so that the first elimination dominates and the later back-substitution phases operate on lower-dimensional systems; we take POSEIDON as the representative example.

Application to POSEIDON. We consider the CICO-3 problems of POSEIDON [42] and POSEIDON2 [43] in sponge mode, using the input/output and internal-subspace models from [44]; precise descriptions of the models and permutation are included in the full version [77]. For the pure I/O model, direct Dixon elimination matches the natural square-system viewpoint. For the mixed-degree subspace models, the Hessenberg bound from Section 3 becomes essential because the polynomial degrees are no longer uniform. Figure 3 compares the resulting complexities for $t = 8$, $c = d = 3$, $\alpha = 3$ across different numbers of partial rounds.

Table 4 reports small-instance timings. Full-round systems behave close to generic dense systems, while partial-round systems are much more structured, so both the Dixon matrix size and the Gröbner-basis degree of regularity fall well below their generic upper bounds. Since the size of this drop depends on the specific subspace model, the generic bound should be read as an upper-bound template rather than as a tight prediction of the best attack. A finer analysis of these structural effects is left for future work.

5.2 Method 2: Iterative elimination

Throughout this section calligraphic letters denote blocks: $\mathcal{F}, \mathcal{P}, \mathcal{R}$ are blocks of polynomials, $\mathcal{X}, \mathcal{Z}$ are blocks of variables, and $|\mathcal{X}|$ is the number of variables in the block $\mathcal{X}$.

Method 2 is designed for ladder-structured systems and generalizes iterative two-polynomial elimination to local multipolynomial subsystems. It can be

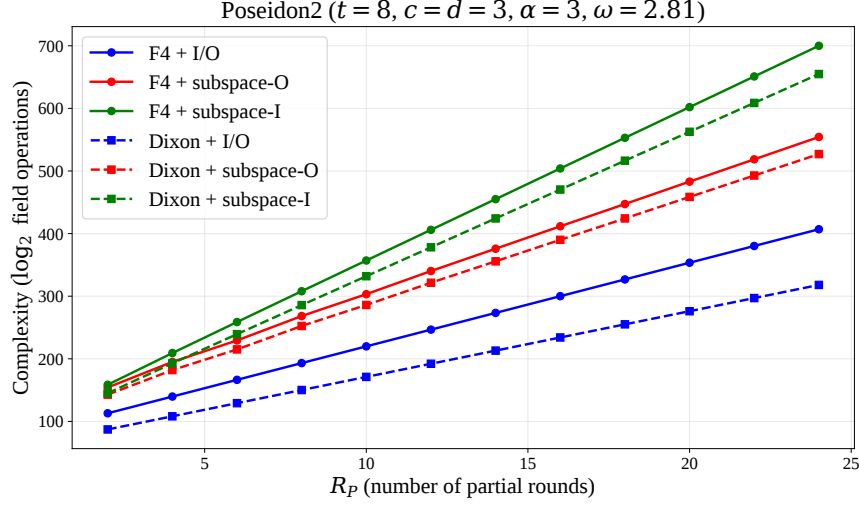


Fig. 3. Complexity comparison for POSEIDON2, with the model parameters $t = 8$, $c = d = 3$, $\alpha = 3$ of [44]. Here $\omega = 2.81$ for all curves.

viewed as a generalization of the resultant-based approach of Yang et al. [84] to multipolynomial Dixon elimination, and as an AO-oriented specialization of hybrid Dixon-matrix construction from [32]. We write such a system as

$$\mathcal{S}: \quad \mathcal{F}_1(\mathcal{X}_0, \mathcal{X}_1), \mathcal{F}_2(\mathcal{X}_1, \mathcal{X}_2), \dots, \mathcal{F}_n(\mathcal{X}_{n-1}, \mathcal{X}_n), \quad (23)$$

with $|\mathcal{F}_1| = |\mathcal{X}_0| + 1$ and $|\mathcal{F}_i| = |\mathcal{X}_{i-1}|$ for $2 \leq i \leq n$. Eliminating successively,

$$R_1 = \text{Res}_{\mathcal{X}_0}(\mathcal{F}_1), R_2 = \text{Res}_{\mathcal{X}_1}(R_1, \mathcal{F}_2), \dots, R_n = \text{Res}_{\mathcal{X}_{n-1}}(R_{n-1}, \mathcal{F}_n), \quad (24)$$

yields a univariate eliminant if $|\mathcal{X}_n| = 1$, or a smaller system otherwise.

Assume each block $\mathcal{F}_j$ has common degree d_j . By Bézout, the ideal-theoretic eliminant produced at step i satisfies

$$\deg(R_i) \leq \prod_{j=1}^i d_j^{|\mathcal{F}_j|}. \quad (25)$$

The last step produces R_n from R_{n-1} and $\mathcal{F}_n$, by eliminating the variables in $\mathcal{X}_{n-1}$. Here R_{n-1} has degree $D_{\text{in}} := \prod_{j=1}^{n-1} d_j^{|\mathcal{F}_j|}$ and each polynomial in $\mathcal{F}_n$ has degree d_n . Viewing this as Dixon elimination on one polynomial of degree D_{in} and $|\mathcal{F}_n|$ polynomials of degree d_n , the Unrestricted Bound (7) gives matrix size at most $\binom{d_n|\mathcal{F}_n| - d_n + D_{\text{in}}}{|\mathcal{F}_n|}$. The total $\mathcal{X}_n$ -degree of the cancellation determinant is at most $D_{\text{in}} + d_n|\mathcal{F}_n|$ (sum of the per-row degrees in the eliminated block), which in turn upper-bounds the entry-wise $\mathcal{X}_n$ -degree of the resulting Step 4

Table 4. Timing comparison for POSEIDON instances ($t = 6, c = 1, d = 3$, I/O modeling, bypass one round) over $\mathbb{F}_p$, $p = 2^{62} + 169$. T and E denote theoretical and experimental values; complexities (Comp.) are in $\log_2$ field operations. Here $\omega = 2.81$. For odd R_F , one additional full round is added at the beginning. Both the Comp. and the timed implementation use the HNF determinant model.

Configuration	Gröbner Basis					Dixon Resultant				
	d_{reg}		Comp.		Time	$D(\mathcal{F})$		Comp.		Time
	T	E	T	E		T	E	T	E	(s)
$R_F = 3, R_P = 0, \alpha = 3$	25	25	32.8	32.8	2.2	117	108	24.1	23.7	0.3
$R_F = 3, R_P = 0, \alpha = 5$	73	71	45.2	44.9	94295	925	875	33.9	33.7	1477
$R_F = 4, R_P = 0, \alpha = 3$	79	79	46.2	46.2	188189	1080	1053	34.7	34.6	1772
$R_F = 2, R_P = 1, \alpha = 3$	25	11	32.8	23.9	0.02	117	36	24.1	19.3	0.006
$R_F = 2, R_P = 1, \alpha = 5$	73	27	45.2	33.7	14.3	925	175	33.9	27.2	2.1
$R_F = 3, R_P = 1, \alpha = 3$	79	31	46.2	35.3	178	1080	891	34.7	33.9	695
$R_F = 2, R_P = 2, \alpha = 3$	79	24	46.2	32.4	0.4	1080	297	34.7	29.4	11.9

polynomial matrix (Section 3.2 Step 4), so HNF gives

$$\tilde{\mathcal{O}}\left((D_{\text{in}} + d_n|\mathcal{F}_n|)\begin{pmatrix} d_n|\mathcal{F}_n| - d_n + D_{\text{in}} \\ |\mathcal{F}_n| \end{pmatrix}^\omega\right), \quad (26)$$

with the Hessenberg Bound (9) available when the Unrestricted Bound is too coarse.

For meet-in-the-middle (MITM) instances, we split the chain into forward and backward parts meeting in a small middle block $\mathcal{Z}$:

$$\mathcal{F}_1(\mathcal{X}_0, \mathcal{X}_1), \dots, \mathcal{F}_n(\mathcal{X}_{n-1}, \mathcal{Z}), \quad \mathcal{F}'_1(\mathcal{Y}_0, \mathcal{Y}_1), \dots, \mathcal{F}'_m(\mathcal{Y}_{m-1}, \mathcal{Z}). \quad (27)$$

Forward and backward eliminations give two middle-state relations of degrees

$$D_{\text{fwd}} = \prod_{j=1}^n d_j^{|\mathcal{F}_j|}, \quad D_{\text{bwd}} = \prod_{j=1}^m d'_j^{|\mathcal{F}'_j|}.$$

The final merge eliminates the middle block $\mathcal{Z}$ and, with $d_{\text{max}} = \max(D_{\text{fwd}}, D_{\text{bwd}})$, admits the coarse bound

$$\tilde{\mathcal{O}}(d_{\text{max}}^{\omega+1}). \quad (28)$$

The complexity of Method 2 is the maximum over all stages: every determinant of the forward and backward chains (26) and the final merge (28). Root finding, back-substitution, and the verification of the candidate solutions in the original system—which also discards the spurious candidates possibly introduced by extraneous factors—are negligible in comparison.

This method is best suited to Type II (equivalence-relation) primitives, whose ladder-structured CICO systems link successive states through local low-degree relations; we take **Vision** as the representative example.

Application to Vision. For **Vision** [4] with $s = 2$, inverse layers make the full direct model unattractive, so we adopt a meet-in-the-middle (MITM) approach following the general framework of Liu et al. [64] and apply iterative Dixon elimination. The full version [77] gives the detailed model, equation distribution, and round function; Figure 4 and Table 5 summarize the outcome here.

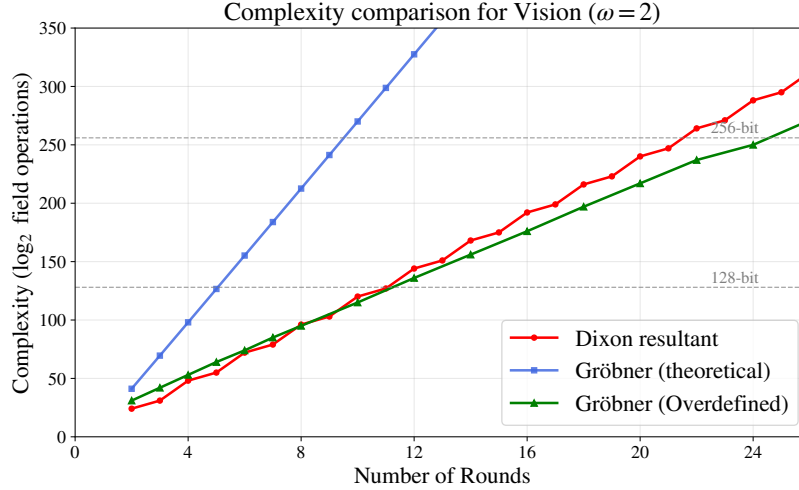


Fig. 4. Complexity comparison for **Vision** ($\omega = 2$ for consistency with [64]).

For these parameters the final merge (28) dominates every forward and backward stage of the chain, so it alone determines the overall complexity. The stage-by-stage figures and a rank bound that keeps the intermediate rungs below the merge are given in the full version [77]. The special structure of **Vision** yields observed degree growth far below the Bézout-based upper bounds. A more precise estimate is left for future work.

Table 5. Experimental timing and complexity comparison for **Vision** ($s = 2$). Complexities (Comp.) are in $\log_2$ field operations. $\omega = 2$ for consistency with [64].

Method	2 rounds		3 rounds		4 rounds	
	Comp.	Time	Comp.	Time	Comp.	Time
Gröbner basis [64]	31.0	1.38s	42.0	1d 4h	53.0	—
Dixon resultant	24.1	0.1s	31.0	15m	48.1	3d

5.3 Method 3: Ideal reduction and hybrid elimination

Method 3 combines Dixon elimination with the reduction machinery developed for FreeLunch-style systems in [14]. We start from a triangular system

$$\mathcal{P} = \begin{cases} z_1^\alpha - f_1(x) = 0, \\ z_2^\alpha - f_2(x, z_1) = 0, \\ \vdots \\ z_n^\alpha - f_n(x, z_1, \dots, z_{n-1}) = 0, \\ h(x, z_1, \dots, z_n) = 0, \end{cases} \quad (29)$$

and use the reduction procedure **Reduce** of [14, Algorithm 1]. For a polynomial g of weighted degree $d_{\mathcal{P}_k}(g)$ in a single retained variable, with $\deg_{z_i} g \leq 2\alpha - 2$, the reduction cost is $\tilde{O}(d_{\mathcal{P}_k}(g)(2\alpha - 1)^n)$ [14, Lemma 1], where n is the number of levels in the triangular ideal $\mathcal{P}_k$. We take the reduced triangular system as input. If it must be constructed from an arithmetic circuit, its construction cost must be added separately [14, Proposition 4]. For CICO-1 and a system that is a FreeLunch Gröbner basis, the optimized two-polynomial strategy of [14, Corollary 3] applies directly and gives

$$\tilde{O}\left(d_T \alpha^2 \left(\frac{2\alpha - 1}{\alpha}\right)^{n+2}\right). \quad (30)$$

For general CICO- k , fix $\alpha \geq 2$ and k , retain $|\mathcal{X}| = k$ input variables, and build k relations $R_j(\mathcal{X})$. Assume $\deg_{z_i} h_j \leq 2\alpha - 2$. Put $B_k(e) = \binom{e+k}{k}$ and $\bar{D} = \max_j d_{\mathcal{P}_k}(h_j)$.

(P1) *Reduction*. Computing the normal forms of the k output constraints costs

$$T_{\text{red}} = \tilde{O}(k B_k(\bar{D}) (2\alpha - 1)^n). \quad (31)$$

(P2) *Successive elimination*. Reduction only lowers the exponents $\deg_{z_i}$ below α ; it does not by itself produce an element of $\mathbb{K}[\mathcal{X}]$. For each constraint, set $h_{j,n} = \text{NF}_{\mathcal{P}_n}(h_j)$ and compute, for $i = n, \dots, 1$, $h_{j,i-1} = \text{NF}_{\mathcal{P}_{i-1}}(\text{Res}_{z_i}(h_{j,i}, z_i^\alpha - f_i))$. Here $\mathcal{P}_i$ contains the first i triangular equations and $R_j = h_{j,0} \in \mathbb{K}[\mathcal{X}]$. With $D_i = \alpha^{n-i} \bar{D}$, the cost is

$$T_{\text{elim}} = \tilde{O}\left(k \sum_{i=1}^n B_k(D_i) (2\alpha - 1)^{i+2}\right). \quad (32)$$

The degree bounds and multivariate cost estimates are proved in the full version [77].

(P3) *Final elimination.* Let $d_{\mathcal{R}} := \max_j \deg(R_j) \leq \alpha^n \bar{D}$. Assuming that $\langle R_1, \dots, R_k \rangle$ is zero-dimensional and the Dixon projection conditions hold, the final step eliminates $k-1$ variables from these k relations by mixed-degree Dixon elimination:

$$T_{\text{final}} = \tilde{O}(\max \{T_1^{\text{best}}(\mathcal{R}), T_3(\mathcal{R}), k d_{\mathcal{R}} D(\mathcal{R})^\omega\}), \quad (33)$$

where T_1^{best} and T_3 are the construction and rank-extraction costs of Section 3.2, and $D(\mathcal{R})$ is bounded by Corollary 1(3), which specializes to $C_k^{(d_{\mathcal{R}})}$ when all the R_j have the same degree.

The total symbolic hybrid cost is, up to a constant factor, the maximum of (31)–(33); in particular the reduction and elimination phases are not absorbed into the final Dixon cost in general. By contrast, direct elimination on the unreduced system of $n+k$ equations gives

$$\tilde{O}\left(\max \left\{T_1^{\text{best}}(\mathcal{P}_k), T_3(\mathcal{P}_k), \left(\sum_i d_i\right) D(\mathcal{P}_k)^\omega\right\}\right), \quad (34)$$

again with $D(\mathcal{P}_k)$ from Corollary 1(3), since the constraints h_j do not have the same degree as the triangular equations; the last term becomes $(n+k)\alpha(C_{n+k}^{(\alpha)})^\omega$ when all equations have degree α . The comparison between the two routes determines whether the reduction phase is worthwhile.

This method is an improved attack on Type II (equivalence-relation) primitives, refining Method 2 by interleaving Dixon elimination with ideal reduction; we take **XHash12** as the representative example.

Remark 9. For the special case of CICO-1, the two-polynomial iterative-resultant route of [14] already gives (30), and Dixon does not change the picture. In concurrent and independent work, Bak et al. [8] develop a related CICO-2 framework, introducing improvements to variable elimination and the final bivariate resultant, with substantial theoretical and practical gains. Although Method 3 does not currently offer a concrete advantage for $k \geq 3$, it provides a more flexible framework for further elimination strategies.

Application to XHash12. We focus on **XHash12** [5]; the full version [77] recalls the permutation and specifies the triangular model used here. Figure 5 compares the direct Dixon method, the hybrid reduction-based strategy, and the corresponding generic Gröbner-basis estimate for each $k = 1, 2, 3, 4$. The hybrid method is excellent for CICO-1 and retains a small margin for CICO-2. For CICO-3 and CICO-4, however, the intermediate degrees make it worse than the direct elimination method.

Table 6 compares symbolic complexity estimates with recorded elimination times for small **XHash** polynomial systems. The special structure of these instances and implementation details can lead to discrepancies between generic complexity bounds and measured running times. A refined complexity analysis accounting for this structure is left for future work.

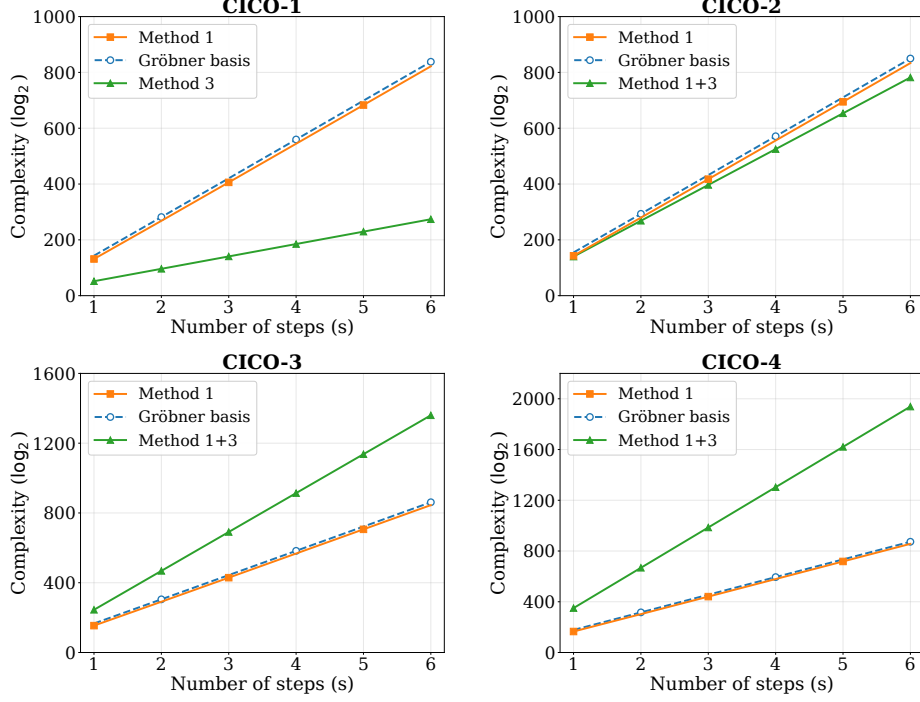


Fig. 5. Symbolic complexity comparison for **XHash12**, $\alpha = 7$ and $\omega = 2.81$, in $\log_2$ field operations. The CICO-1 Method 3 curve uses (30); the hybrid Method 1+3 for $k \geq 2$ includes reduction, successive elimination and final Dixon elimination. Each GB curve uses $\binom{m+d_{\text{reg}}}{m}^\omega$, where $m = 12s + k$ and $d_{\text{reg}} = m(\alpha - 1) + 1$.

Table 6. Elimination times and symbolic complexity estimates for small **XHash** polynomial systems. The tuples specify $(\alpha, \text{State}, \text{step})$: the exponent, state width, and number of steps. Comp. denotes $\log_2$ field operations with $\omega = 2.81$.

	CICO-1 (3, 3, 5)		CICO-1 (7, 3, 5)		CICO-2 (3, 4, 3)	
Method	Comp.	Time	Comp.	Time	Comp.	Time
Gröbner basis	48.8	1.5s	74.7	-	41.4	59.7s
Method 1	42.6	5.12s	64.4	-	35.1	4.5s
Method 1+3	18.6	0.04s	29.6	200.4s	31.2	413.2s

6 Conclusion

In this work, we revisited the Dixon resultant as a practical and theoretically grounded tool for algebraic cryptanalysis. By establishing upper bounds on the Dixon matrix size that are attained by the monomial support of generic systems, we derived refined complexity estimates. Combined with an efficient implemen-

tation and a degree-aware submatrix selection strategy, our framework enables reliable elimination for polynomial systems arising in cryptographic applications.

Our results show that Dixon resultants and Gröbner basis methods play complementary roles in algebraic cryptanalysis. In well-determined settings, Dixon resultants are comparable to Gröbner bases. Gröbner basis methods are more effective for overdetermined systems, whereas Dixon resultants provide a structured determinant-based elimination subroutine for underdetermined settings.

There remain several directions for future work.

1. Further analyzing AO primitives through better algebraic models for Dixon elimination, refined complexity analyses accounting for AO-specific structures, and applications to Type III (lookup-table) primitives in combination with recent algebraic techniques [9];
2. Improving Dixon-resultant techniques, including rank estimation to refine Step 4 complexity estimates and optimize Step 3, the exploitation of overdetermined systems to reduce complexity, nonheuristic extraneous-factor elimination, and connections with multivariate subresultants [30];
3. Optimizing Dixon resultants for MQ systems, with potential applications to schemes such as UOV [57]. Although asymptotically favorable under the compared naive Gröbner-basis bounds, direct Dixon elimination is generally slower than Magma in our MQ experiments and theoretically less efficient than guessing-based hybrid methods [21]. Improving its theoretical or practical performance on MQ systems is a direction for future work.

Acknowledgments. We would like to thank our advisors, Meiqin Wang and Kai Hu, for their guidance and support. We are grateful to all the anonymous reviewers for their valuable comments and suggestions. We thank Biran Lu for an elegant argument on the monomial-counting problem that inspired the proof in Section 3.1, particularly Lemma 2. We also thank Robert H. Lewis, Michael Monagan, Ayoola Jinadu, and Guoqiang Deng for their helpful correspondence regarding the implementation of the Dixon resultant.

This work is supported by the National Cryptologic Science Fund of China (Grant No. 2025NCSF01013), the National Key R&D Program of China (Grant No. 2024YFA1013000), the National Natural Science Foundation of China (Grant No. U2336207), the Key R&D Program of Shandong Province, China (Grant No. 2026CXPT037), and the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (Grant No. JYB2025XDXM114).

Claude and ChatGPT were used to assist with the implementation of the DRSolve software and the language polishing of this paper. GPT-6 Astra was also used to review the mathematical derivations in the final version and provide revision suggestions. All AI-assisted revisions to the paper were reviewed by the authors, and all AI-assisted code was tested by the authors. The authors take full responsibility for the paper and the DRSolve software.

References

1. Albrecht, M.R., Grassi, L., Perrin, L., Ramacher, S., Rechberger, C., Rotaru, D., Roy, A., Schofnegger, M.: Feistel structures for MPC, and more. In: ESORICS

2019. Lecture Notes in Computer Science, vol. 11736, pp. 151–171. Springer (2019). https://doi.org/10.1007/978-3-030-29962-0_8
2. Albrecht, M.R., Grassi, L., Rechberger, C., Roy, A., Tiessen, T.: MiMC: Efficient encryption and cryptographic hashing with minimal multiplicative complexity. In: ASIACRYPT 2016. Lecture Notes in Computer Science, vol. 10031, pp. 191–219 (2016). https://doi.org/10.1007/978-3-662-53887-6_7
3. Albrecht, M.: Algebraic attacks on the Courtois toy cipher. *Cryptologia* **32**(3), 220–276 (2008). <https://doi.org/10.1080/0161190802058139>
4. Aly, A., Ashur, T., Ben-Sasson, E., Dhooghe, S., Szeponiec, A.: Design of symmetric-key primitives for advanced cryptographic protocols. *IACR Trans. Symmetric Cryptol.* **2020**(3), 1–45 (2020). <https://doi.org/10.13154/tosc.v2020.i3.1-45>
5. Ashur, T., Al Kindi, Mahzoun, M.: XHash8 and XHash12: Efficient STARK-friendly hash functions. *IACR Cryptol. ePrint Arch.* p. 1045 (2023), <https://eprint.iacr.org/2023/1045>
6. Aval, J.: Multivariate Fuss-Catalan numbers. *Discret. Math.* **308**(20), 4660–4669 (2008). <https://doi.org/10.1016/J.DISC.2007.08.100>
7. Bak, A., Bariant, A., Boeuf, A., Briaud, P., Øygarden, M., Phanse, A.: The algebraic CheapLunch: Extending FreeLunch attacks on arithmetization-oriented primitives beyond CICO-1. *IACR Cryptol. ePrint Arch.* p. 2040 (2025), <https://eprint.iacr.org/2025/2040>
8. Bak, A., Bariant, A., Hostettler, M., Neiger, V.: Resultants meet resultant: Improving CICO-1 and CICO-2 attacks on ZK-friendly permutations. *IACR Cryptol. ePrint Arch.* **2026**, 1281 (2026), <https://eprint.iacr.org/2026/1281>
9. Bak, A., Jazeran, G., Galissant, P., Perrin, L.: Attacking split-and-lookup-based primitives using probabilistic polynomial system solving applications to round-reduced Monolith and full-round Skyscraper. *IACR Trans. Symmetric Cryptol.* **2025**(3), 337–367 (2025). <https://doi.org/10.46586/TOSC.V2025.I3.337-367>
10. Bardet, M., Faugère, J.C., Salvy, B.: On the complexity of Gröbner basis computation of semi-regular overdetermined algebraic equations. In: Proceedings of the International Conference on Polynomial System Solving (ICPSS). pp. 71–74 (2004)
11. Bardet, M., Faugère, J., Salvy, B.: On the complexity of the F5 Gröbner basis algorithm. *J. Symb. Comput.* **70**, 49–70 (2015). <https://doi.org/10.1016/J.JSC.2014.09.025>
12. Bardet, M., Faugère, J.C., Salvy, B., Yang, B.Y.: Asymptotic behaviour of the degree of regularity of semi-regular polynomial systems. In: Proc. of MEGA. vol. 5, pp. 2–2 (2005)
13. Bareiss, E.H.: Sylvester’s identity and multistep integer-preserving Gaussian elimination. *Mathematics of Computation* **22**(103), 565–578 (1968). <https://doi.org/10.1090/S0025-5718-1968-0226829-0>
14. Bariant, A., Boeuf, A., Briaud, P., Hostettler, M., Øygarden, M., Raddum, H.: Improved resultant attack against arithmetization-oriented primitives. In: Kalai, Y.T., Kamara, S.F. (eds.) *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference*, Santa Barbara, CA, USA, August 17–21, 2025, Proceedings, Part V. Lecture Notes in Computer Science, vol. 16004, pp. 335–367. Springer (2025). https://doi.org/10.1007/978-3-032-01901-1_11
15. Bariant, A., Boeuf, A., Lemoine, A., Ayala, I.M., Øygarden, M., Perrin, L., Raddum, H.: The algebraic FreeLunch: Efficient Gröbner basis attacks against arithmetization-oriented primitives. In: Reyzin, L., Stebila, D. (eds.) *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference*, Santa Barbara, CA, USA, August 18–22, 2024, Proceedings, Part IV.

- Lecture Notes in Computer Science, vol. 14923, pp. 139–173. Springer (2024). https://doi.org/10.1007/978-3-031-68385-5_5
16. Bariant, A., Bouvier, C., Leurent, G., Perrin, L.: Algebraic attacks against some arithmetization-oriented primitives. *IACR Trans. Symmetric Cryptol.* **2022**(3), 73–101 (2022). <https://doi.org/10.46586/TOSC.V2022.I3.73-101>
 17. Berthomieu, J., Eder, C., Safey El Din, M.: msolve: A library for solving polynomial systems. In: Proceedings of the 2021 on International Symposium on Symbolic and Algebraic Computation. pp. 51–58. ISSAC '21, ACM (2021). <https://doi.org/10.1145/3452143.3465545>
 18. Berthomieu, J., Neiger, V., Safey El Din, M.: Faster change of order algorithm for Gröbner bases under shape and stability assumptions. In: Proceedings of the 2022 International Symposium on Symbolic and Algebraic Computation, ISSAC 2022. pp. 409–418. ACM (2022). <https://doi.org/10.1145/3476446.3535484>, <https://hal.sorbonne-universite.fr/hal-03580736>
 19. Bertoni, G., Daemen, J., Peeters, M., Van Assche, G.: Sponge functions. In: ECRYPT Hash Workshop (2007)
 20. Bertoni, G., Daemen, J., Peeters, M., Van Assche, G.: On the indifferentiability of the sponge construction. In: Smart, N.P. (ed.) *Advances in Cryptology - EUROCRYPT 2008*, 27th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Istanbul, Turkey, April 13–17, 2008. Proceedings. Lecture Notes in Computer Science, vol. 4965, pp. 181–197. Springer (2008). https://doi.org/10.1007/978-3-540-78967-3_11
 21. Bettale, L., Faugère, J., Perret, L.: Hybrid approach for solving multivariate systems over finite fields. *J. Math. Cryptol.* **3**(3), 177–197 (2009). <https://doi.org/10.1515/JMC.2009.009>
 22. Bóna, M. (ed.): *Handbook of Enumerative Combinatorics*. Discrete Mathematics and Its Applications, CRC Press (2015), <https://www.routledge.com/Handbook-of-Enumerative-Combinatorics/Bona/p/book/9781482220858>
 23. Bosma, W., Cannon, J., Playoust, C.: The Magma algebra system I: The user language. *Journal of Symbolic Computation* **24**(3–4), 235–265 (1997). <https://doi.org/10.1006/jsco.1996.0125>
 24. Bouvier, C.: Cryptanalysis and design of symmetric primitives defined over large finite fields. (Cryptanalyse et conception de primitives symétriques définies sur de grands corps finis). Ph.D. thesis, Sorbonne University, Paris, France (2023), <https://tel.archives-ouvertes.fr/tel-04327955>
 25. Bouvier, C., Briaud, P., Chaidos, P., Perrin, L., Salen, R., Velichkov, V., Willems, D.: New design techniques for efficient arithmetization-oriented hash functions: Anemoi permutations and Jive compression mode. In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023*, Santa Barbara, CA, USA, August 20–24, 2023, Proceedings, Part III. Lecture Notes in Computer Science, vol. 14083, pp. 507–539. Springer (2023). https://doi.org/10.1007/978-3-031-38548-3_17
 26. Cahill, N.D., D’Errico, J.R., Narayan, D.A., Narayan, J.Y.: Fibonacci determinants. *The College Mathematics Journal* **33**(3), 221–225 (2002)
 27. Chtcherba, A.D., Kapur, D.: Constructing Sylvester-type resultant matrices using the Dixon formulation. *Journal of Symbolic Computation* **38**(1), 777–814 (2004). <https://doi.org/10.1016/j.jsc.2003.11.003>
 28. Chtcherba, A.D., Kapur, D.: Resultants for unmixed bivariate polynomial systems produced using the Dixon formulation. *Journal of Symbolic Computation* **38**(2), 915–958 (2004). <https://doi.org/10.1016/j.jsc.2003.12.001>

29. Collart, S., Kalkbrener, M., Mall, D.: Converting bases with the Gröbner walk. *J. Symb. Comput.* **24**(3/4), 465–469 (1997). <https://doi.org/10.1006/JSC0.1996.0145>
30. Cox, D., D’Andrea, C.: Subresultants and the shape lemma. *Math. Comput.* **92**(343), 2355–2379 (2023). <https://doi.org/10.1090/MCOM/3840>
31. Cox, D.A., Little, J., O’Shea, D.: *Using Algebraic Geometry*. Graduate Texts in Mathematics, Springer New York, 2 edn. (2005), <https://books.google.com.tw/books?id=1blxiz0S9N0C>
32. Deng, G., Qi, N., Tang, M., Duan, X.: Constructing Dixon matrix for sparse polynomial equations based on hybrid and heuristics scheme. *Symmetry* **14**(6), 1174 (2022). <https://doi.org/10.3390/sym14061174>
33. Dixon, A.L.: The eliminant of three quantics in two independent variables. *Proceedings of the London Mathematical Society* **s2-7**(1), 49–69 (1909). <https://doi.org/10.1112/plms/s2-7.1.49>
34. Faugère, J.C.: A new efficient algorithm for computing Gröbner bases (F4). *Journal of Pure and Applied Algebra* **139**(1-3), 61–88 (1999)
35. Faugère, J.C.: A new efficient algorithm for computing Gröbner bases without reduction to zero (F5). In: *Proceedings of the 2002 International Symposium on Symbolic and Algebraic Computation*. pp. 75–83. ISSAC ’02, Association for Computing Machinery, New York, NY, USA (2002). <https://doi.org/10.1145/780506.780516>
36. Faugère, J., Gianni, P.M., Lazard, D., Mora, T.: Efficient computation of zero-dimensional Gröbner bases by change of ordering. *J. Symb. Comput.* **16**(4), 329–344 (1993). <https://doi.org/10.1006/JSC0.1993.1051>
37. Fulton, W.: *Intersection Theory*. *Ergebnisse der Mathematik und ihrer Grenzgebiete*, Springer, 2 edn. (1998). <https://doi.org/10.1007/978-1-4612-1700-8>
38. Gentleman, W.M., Johnson, S.C.: Analysis of algorithms, a case study: Determinants of matrices with polynomial entries. *ACM Transactions on Mathematical Software* **2**(3), 232–241 (1976)
39. Grassi, L., Hao, Y., Rechberger, C., Schofnegger, M., Walch, R., Wang, Q.: Horst meets fluid-SPN: Griffin for zero-knowledge applications. In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology - CRYPTO 2023*. *Lecture Notes in Computer Science*, vol. 14083, pp. 573–606. Springer (2023). https://doi.org/10.1007/978-3-031-38548-3_19
40. Grassi, L., Khovratovich, D., Lüftenegger, R., Rechberger, C., Schofnegger, M., Walch, R.: Reinforced Concrete: A fast hash function for verifiable computation. In: *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022*. pp. 1323–1335. ACM (2022). <https://doi.org/10.1145/3548606.3560686>
41. Grassi, L., Khovratovich, D., Lüftenegger, R., Rechberger, C., Schofnegger, M., Walch, R.: Monolith: Circuit-friendly hash functions with new nonlinear layers for fast and constant-time implementations. *IACR Trans. Symmetric Cryptol.* **2024**(3), 44–83 (2024). <https://doi.org/10.46586/tosc.v2024.i3.44-83>, <https://eprint.iacr.org/2023/1025>
42. Grassi, L., Khovratovich, D., Rechberger, C., Roy, A., Schofnegger, M.: Poseidon: A new hash function for zero-knowledge proof systems. In: Bailey, M.D., Greenstadt, R. (eds.) *30th USENIX Security Symposium, USENIX Security 2021, August 11–13, 2021*. pp. 519–535. USENIX Association (2021), <https://www.usenix.org/conference/usenixsecurity21/presentation/grassi>

43. Grassi, L., Khovratovich, D., Schofnegger, M.: Poseidon2: A faster version of the Poseidon hash function. In: Mrabet, N.E., Feo, L.D., Duquesne, S. (eds.) Progress in Cryptology - AFRICACRYPT 2023 - 14th International Conference on Cryptology in Africa, Sousse, Tunisia, July 19-21, 2023, Proceedings. Lecture Notes in Computer Science, vol. 14064, pp. 177–203. Springer (2023). https://doi.org/10.1007/978-3-031-37679-5_8
44. Grassi, L., Koschatko, K., Rechberger, C.: Poseidon and Neptune: Gröbner basis cryptanalysis exploiting subspace trails. IACR Trans. Symmetric Cryptol. **2025**(2), 34–86 (2025). <https://doi.org/10.46586/TOSC.V2025.I2.34-86>
45. Grassi, L., Onofri, S., Pedicini, M., Sozzi, L.: Invertible quadratic non-linear layers for MPC-/FHE-/ZK-friendly schemes over $\mathbb{F}_p^n$ application to Poseidon. IACR Trans. Symmetric Cryptol. **2022**(3), 20–72 (2022). <https://doi.org/10.46586/TOSC.V2022.I3.20-72>
46. Heintz, J.: Definability and fast quantifier elimination in algebraically closed fields. Theor. Comput. Sci. **24**, 239–277 (1983). [https://doi.org/10.1016/0304-3975\(83\)90002-6](https://doi.org/10.1016/0304-3975(83)90002-6)
47. Hilton, P., Pedersen, J.: Catalan numbers, their generalization, and their uses. The Mathematical Intelligencer **13**(2), 64–75 (1991)
48. Horowitz, E., Sahni, S.: On computing the exact determinant of matrices with polynomial entries. Journal of the ACM **22**(1), 38–50 (1975). <https://doi.org/10.1145/321864.321868>
49. Huang, Q.: Sparse polynomial interpolation based on derivatives. J. Symb. Comput. **114**, 359–375 (2023). <https://doi.org/10.1016/J.JSC.2022.06.002>
50. Hyun, S.G., Neiger, V., Schost, É.: Implementations of efficient univariate polynomial matrix algorithms and application to bivariate resultants. In: Proceedings ISSAC 2019. pp. 235–242. ACM (2019). <https://doi.org/10.1145/3326229.3326272>, <https://github.com/vneiger/pml>
51. Jinadu, A., Monagan, M.: A new Dixon resultant algorithm for solving parametric polynomial systems using sparse multivariate rational function interpolation. Preprint (2025), <https://www.cec.m.sfu.ca/CAG/papers/AyoolaJSC25.pdf>
52. Jinadu, A., Monagan, M.B.: An interpolation algorithm for computing Dixon resultants. In: Boulier, F., England, M., Sadykov, T.M., Vorozhtsov, E.V. (eds.) Computer Algebra in Scientific Computing - 24th International Workshop, CASC 2022, Gebze, Turkey, August 22-26, 2022, Proceedings. Lecture Notes in Computer Science, vol. 13366, pp. 185–205. Springer (2022). https://doi.org/10.1007/978-3-031-14788-3_11
53. Jinadu, A.I.: Solving Parametric Systems Using Dixon Resultants and Sparse Interpolation Tools. Ph.D. thesis, Simon Fraser University (Summer 2023)
54. Kapur, D., Saxena, T.: Sparsity considerations in Dixon resultants. In: Miller, G.L. (ed.) Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing, Philadelphia, Pennsylvania, USA, May 22-24, 1996. pp. 184–191. ACM (1996). <https://doi.org/10.1145/237814.237862>
55. Kapur, D., Saxena, T.: Extraneous factors in the Dixon resultant formulation. In: Char, B.W., Wang, P.S., Küchlin, W. (eds.) Proceedings of the 1997 International Symposium on Symbolic and Algebraic Computation, ISSAC 1997, Maui, Hawaii, USA, July 21-23, 1997. pp. 141–148. ACM (1997). <https://doi.org/10.1145/258726.258768>
56. Kapur, D., Saxena, T., Yang, L.: Algebraic and geometric reasoning using Dixon resultants. In: MacCallum, M.A.H. (ed.) Proceedings of the International Symposium on Symbolic and Algebraic Computation, ISSAC '94, Oxford, UK, July 20-22, 1994. pp. 99–107. ACM (1994). <https://doi.org/10.1145/190347.190372>

57. Kipnis, A., Patarin, J., Goubin, L.: Unbalanced oil and vinegar signature schemes. In: Stern, J. (ed.) *Advances in Cryptology - EUROCRYPT '99*, International Conference on the Theory and Application of Cryptographic Techniques, Prague, Czech Republic, May 2-6, 1999, Proceeding. Lecture Notes in Computer Science, vol. 1592, pp. 206–222. Springer (1999). https://doi.org/10.1007/3-540-48910-X_15
58. Koschatko, K., Lüftenegger, R., Rechberger, C.: Exploring the six worlds of Gröbner basis cryptanalysis: Application to Anemoi. *IACR Trans. Symmetric Cryptol.* **2024**(4), 138–190 (2024). <https://doi.org/10.46586/TOSC.V2024.I4.138-190>
59. Kouba, R., Neiger, V., Din, M.S.E.: A complexity analysis of the f4 gröbner basis algorithm with tracer data (2026), <https://arxiv.org/abs/2603.16378>
60. Labahn, G., Neiger, V., Zhou, W.: Fast, deterministic computation of the Hermite normal form and determinant of a polynomial matrix. *J. Complex.* **42**, 44–71 (2017). <https://doi.org/10.1016/J.JCO.2017.03.003>
61. Lewis, R.H.: Dixon-EDF: The premier method for solution of parametric polynomial systems. In: Kotsireas, I.S., Martínez-Moro, E. (eds.) *Applications of Computer Algebra*, Springer Proceedings in Mathematics & Statistics, vol. 198, pp. 237–256. Springer International Publishing, Cham (2017). https://doi.org/10.1007/978-3-319-56932-1_16
62. Lewis, R.H.: Fermat computer algebra system. <https://home.bway.net/lewis/> (2026), version 7.9b
63. Li, Y.: An effective algorithm of computing symbolic determinants with multivariate polynomial entries. *Applied Mathematics and Computation* **192**(2), 382–388 (2007)
64. Liu, F., Mahzoun, M., Meier, W.: Modelling ciphers with overdefined systems of quadratic equations: Application to friday, vision, RAIN and biscuit. In: Chung, K., Sasaki, Y. (eds.) *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security*, Kolkata, India, December 9-13, 2024, Proceedings, Part VII. Lecture Notes in Computer Science, vol. 15490, pp. 424–456. Springer (2024). https://doi.org/10.1007/978-981-96-0941-3_14
65. Macaulay, F.S.: Some formulae in elimination. *Proceedings of the London Mathematical Society* **s1-35**(1), 3–27 (1902). <https://doi.org/10.1112/plms/s1-35.1.3>
66. Meurant, G.: *Hessenberg and Tridiagonal Matrices: Theory and Examples*. SIAM (2025), <https://epubs.siam.org/doi/10.1137/1.9781611978452>
67. Painvin, L.: Note sur la méthode d'élimination de Bézout. *Nouvelles annales de mathématiques: journal des candidats aux écoles polytechnique et normale* **13**, 278–285 (1874)
68. Qin, X., Wu, D., Tang, L., Ji, Z.: Complexity of constructing Dixon resultant matrix. *Int. J. Comput. Math.* **94**(10), 2074–2088 (2017). <https://doi.org/10.1080/00207160.2016.1276572>
69. Qin, X., Zhang, L., Yang, L., Cao, S.: Heuristics to sift extraneous factors in Dixon resultants. *J. Symb. Comput.* **112**, 105–121 (2022). <https://doi.org/10.1016/J.JSC.2022.01.003>
70. Roy, A., Steiner, M.J., Trevisani, S.: Arion: Arithmetization-oriented permutation and hashing from generalized triangular dynamical systems. *CoRR abs/2303.04639* (2023). <https://doi.org/10.48550/ARXIV.2303.04639>
71. Sauer, J.F., Szepieniec, A.: SoK: Gröbner basis algorithms for arithmetization oriented ciphers. *IACR Cryptol. ePrint Arch.* p. 870 (2021), <https://eprint.iacr.org/2021/870>

72. Schwartz, J.T.: Fast probabilistic algorithms for verification of polynomial identities. *Journal of the ACM (JACM)* **27**(4), 701–717 (1980)
73. Spaenlehauer, P.: Solving multi-homogeneous and determinantal systems: algorithms, complexity, applications. (Résolution de systèmes multi-homogènes et déterminantiels : algorithmes, complexité, applications). Ph.D. thesis, Pierre and Marie Curie University, Paris, France (2012), <https://tel.archives-ouvertes.fr/tel-01110756>
74. Steiner, M.J.: Solving degree bounds for iterated polynomial systems. *IACR Trans. Symmetric Cryptol.* **2024**(1), 357–411 (2024). <https://doi.org/10.46586/TOSC.V2024.I1.357-411>
75. Storjohann, A.: Algorithms for matrix canonical forms. Ph.D. thesis, ETH Zurich (2000)
76. Sturmfels, B.: Solving Systems of Polynomial Equations. Conference Board of the Mathematical Sciences Regional Conference Series in Mathematics, Conference Board of the Mathematical Sciences (2002), <https://books.google.com.tw/books?id=WUAjMQAACAJ>
77. Suo, H., Cui, J.: Efficient polynomial system solving via dixon resultants: Applications to AO primitives. *Cryptology ePrint Archive*, Paper 2026/1969 (2026), <https://eprint.iacr.org/2026/1969>
78. Szepeieniec, A., Lemmens, A., Sauer, J.F., Threadbare, B., Al-Kindi: The Tip5 hash function for recursive STARKs. *Cryptology ePrint Archive*, Paper 2023/107 (2023), <https://eprint.iacr.org/2023/107>
79. Tang, X., Feng, Y.: A new efficient algorithm for solving systems of multivariate polynomial equations. *Cryptology ePrint Archive*, Paper 2005/312 (2005), <https://eprint.iacr.org/2005/312>
80. The FLINT team: FLINT: Fast Library for Number Theory (2026), version 3.6.0, <https://flintlib.org>
81. The PML team: PML: Polynomial Matrix Library (2025), version 0.5, <https://github.com/vneiger/pml>
82. Villard, G.: On computing the resultant of generic bivariate polynomials. In: Proceedings of the 2018 International Symposium on Symbolic and Algebraic Computation, ISSAC 2018. pp. 391–398. ACM (2018). <https://doi.org/10.1145/3208976.3209020>
83. Wang, D.: Solving polynomial equations: Characteristic sets and triangular systems. *Mathematics and Computers in Simulation* **42**(4-6), 339–351 (1996). [https://doi.org/10.1016/S0378-4754\(96\)00008-0](https://doi.org/10.1016/S0378-4754(96)00008-0)
84. Yang, H., Zheng, Q., Yang, J., Liu, Q., Tang, D.: A new security evaluation method based on resultant for arithmetic-oriented algorithms. In: Chung, K., Sasaki, Y. (eds.) *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security*, Kolkata, India, December 9-13, 2024, Proceedings, Part VII. *Lecture Notes in Computer Science*, vol. 15490, pp. 457–489. Springer (2024). https://doi.org/10.1007/978-981-96-0941-3_15
85. Zhao, S., Fu, H.: An extended fast algorithm for constructing the Dixon resultant matrix. *Science in China Series A: Mathematics* **48**(1), 131–143 (2005)
86. Zippel, R.: Probabilistic algorithms for sparse polynomials. In: *International symposium on symbolic and algebraic manipulation*. pp. 216–226. Springer (1979)